

面向越南留学生的铁道专业汉越术语翻译学习软件V1.0源代码

一、源代码属性结构说明

- 软件名称：面向越南留学生的铁道专业汉越术语翻译学习软件V1.0
- 源码根目录：当前项目根目录，源码稿仅列出相对路径，不暴露真实服务器路径。
- 选入范围：自研前端页面、后端服务、数据库 SQL、配置文件、业务测试脚本和业务运行脚本。排除第三方依赖、构建产物、锁文件、图片、缓存、文档输出目录以及与目标软件运行无关的材料辅助工具。
- 脱敏说明：数据库口令、令牌密钥、账号密码、邮箱、手机号、主机地址和接口地址已统一脱敏为 *** 或等价占位值。

二、三级目录结构

```
README.md
client/index.html
client/package.json
client/src/App.vue
client/src/api
client/src/assets
client/src/locales
client/src/main.js
client/vite.config.js
package.json
server/.env.example
server/package.json
server/scripts/seed-test-data.js
server/scripts/smoke-test.js
server/src/config
server/src/db
server/src/index.js
server/src/middleware
server/src/routes
server/src/utils
```

三、逐文件源码正文

3.1 README.md

文件路径：README.md

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
# chinese-vietnamese-railway-term-learning

面向越南留学生的铁道专业汉越术语翻译学习软件 V1.0

H5 前端 + Node.js 后台 + MySQL 数据库的铁道专业术语翻译学习平台。

## 功能范围

- 三语言界面：中文、English、Tiếng Việt
- 专业术语管理台：分类、三语术语、难度、释义、例句、启停用
- 练习平台：按分类/难度生成单选、判断、填空、翻译练习
- 数据分析：学习次数、正确率、错题、高频错误术语、分类统计
- 公共系统模块：登录、角色、用户、登录日志、操作日志

## 数据库配置

默认读取 `server/.env`：
```

DB_HOST=*** DB_PORT=3306 DB_USER=root DB_PASSWORD=*** DB_NAME=railway_term_learning JWT_SECRET=***

首次启动后端会自动创建数据表和演示数据。请先确保 MySQL 已启动，并且 `root` 账号允许从本机连接。

启动

npm install npm run dev

- 前端: `http://***:5173`
- 后端: `http://***:3000`

演示账号:

- 管理员: `admin / ***`
- 教师: `teacher / ***`
- 学生: `student / ***`

开源协议

本项目使用 GNU General Public License v2.0 (GPL-2.0) 开源，详见 `LICENSE`。

3.2 client/index.html

文件路径: `client/index.html`

文件职责: 该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文:

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" href="/favicon.svg" />
    <title>汉越铁道术语学习软件</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

3.3 client/package.json

文件路径: `client/package.json`

文件职责: 该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文:

```
{
  "name": "railway-term-learning-client",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build",
    "preview": "vite preview --host 0.0.0.0"
  },
  "dependencies": {
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.7",
    "vue": "^3.5.13"
  },
  "devDependencies": {}
}
```

文件路径：client/src/App.vue

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
<script setup>
import { computed, onMounted, reactive, ref } from 'vue';
import { clearToken, request, setToken } from './api/http';
import { messages } from './locales/messages';

const lang = ref(localStorage.getItem('lang') || 'zh');
const page = ref('dashboard');
const loading = ref(false);
const error = ref('');
const loginForm = reactive({ username: 'admin', password: '***' });
const user = ref(JSON.parse(localStorage.getItem('user') || 'null'));
const terms = ref([]);
const categories = ref([]);
const users = ref([]);
const roles = ref([]);
const loginLogs = ref([]);
const adminAnalysis = ref(null);
const studentAnalysis = ref(null);
const wrongQuestions = ref([]);
const filters = reactive({ keyword: '', categoryId: '', difficulty: '' });
const termPager = reactive({ page: 1, pageSize: 10, total: 0 });
const userPager = reactive({ page: 1, pageSize: 10, total: 0 });
const logPager = reactive({ page: 1, pageSize: 10, total: 0 });
const termForm = reactive(emptyTerm());
const editingId = ref(null);
const showTermForm = ref(false);
const practiceConfig = reactive({ type: 'choice', categoryId: '', difficulty: '', count: 5 });
const questions = ref([]);
const answers = reactive({});
const practiceResult = ref(null);

const t = computed(() => messages[lang.value]);
const isLoggedIn = computed(() => Boolean(user.value));
const canManage = computed(() => ['admin', 'teacher'].includes(user.value?.roleCode || user.value?.role_code));
const statsCards = computed(() => [
  { label: t.value.totalTerms, value: adminAnalysis.value?.summary?.terms ?? terms.value.length },
  { label: t.value.practiceCount, value: studentAnalysis.value?.summary?.practice_count ?? 0 },
  { label: t.value.avgScore, value: studentAnalysis.value?.summary?.avg_score ?? 0 },
  { label: t.value.correctRate, value: rateText.value }
]);
const rateText = computed(() => {
  const s = studentAnalysis.value?.summary;
  if (!s || Number(s.answer_count) === 0) return '0%';
  return `${Math.round((Number(s.correct_count) / Number(s.answer_count)) * 100)}%`;
});

function emptyTerm() {
  return {
    category_id: '',
    term_zh: '',
    term_en: '',
    term_vi: '',
    pinyin: '',
    difficulty: 'easy',
    definition_zh: '',
    definition_en: '',
    definition_vi: '',
    example_zh: '',
    example_en: '',
    example_vi: '',
    status: 'enabled'
  };
};
```

```

}

function setLanguage(value) {
  lang.value = value;
  localStorage.setItem('lang', value);
}

async function login() {
  error.value = '';
  loading.value = true;
  try {
    const data = await request('/auth/login', { method: 'POST', body: loginForm });
    setToken(data.token);
    user.value = data.user;
    localStorage.setItem('user', JSON.stringify(data.user));
    await bootstrap();
  } catch (err) {
    error.value = err.message;
  } finally {
    loading.value = false;
  }
}

function logout() {
  clearToken();
  user.value = null;
  questions.value = [];
  practiceResult.value = null;
}

async function bootstrap() {
  await Promise.all([loadCategories(), loadTerms(), loadAnalysis(), loadWrongQuestions()]);
  if (canManage.value) await Promise.all([loadUsers(), loadRoles(), loadLogs()]);
}

async function loadCategories() {
  categories.value = await request('/terms/categories');
}

async function loadTerms() {
  const params = new URLSearchParams();
  if (filters.keyword) params.set('keyword', filters.keyword);
  if (filters.categoryId) params.set('categoryId', filters.categoryId);
  if (filters.difficulty) params.set('difficulty', filters.difficulty);
  params.set('page', termPager.page);
  params.set('pageSize', termPager.pageSize);
  const data = await request(`/terms?${params.toString()}`);
  terms.value = data.rows || data;
  termPager.total = data.total ?? terms.value.length;
}

async function loadUsers() {
  const data = await request(`/system/users?page=${userPager.page}&pageSize=${userPager.pageSize}`);
  users.value = data.rows || data;
  userPager.total = data.total ?? users.value.length;
}

async function loadRoles() {
  roles.value = await request('/system/roles');
}

async function loadLogs() {
  const data = await request(`/system/logs/login?page=${logPager.page}&pageSize=${logPager.pageSize}`);
  loginLogs.value = data.rows || data;
  logPager.total = data.total ?? loginLogs.value.length;
}

async function loadAnalysis() {
  studentAnalysis.value = await request('/analysis/student');
  adminAnalysis.value = await request('/analysis/admin');
}

```

```

}

async function loadWrongQuestions() {
  const data = await request('/practice/wrong-questions?page=1&pageSize=20');
  wrongQuestions.value = data.rows || data;
}

function resetTermSearch() {
  termPager.page = 1;
  loadTerms();
}

function pageCount(pager) {
  return Math.max(Math.ceil(pager.total / pager.pageSize), 1);
}

async function changePage(pager, nextPage, loader) {
  pager.page = Math.min(Math.max(nextPage, 1), pageCount(pager));
  await loader();
}

function newTerm() {
  Object.assign(termForm, emptyTerm(), { category_id: categories.value[0]?.id || '' });
  editingId.value = null;
  showTermForm.value = true;
}

function editTerm(term) {
  Object.assign(termForm, term, { category_id: term.category_id });
  editingId.value = term.id;
  showTermForm.value = true;
}

async function saveTerm() {
  const path = editingId.value ? `/terms/${editingId.value}` : '/terms';
  const method = editingId.value ? 'PUT' : 'POST';
  await request(path, { method, body: termForm });
  showTermForm.value = false;
  await Promise.all([loadTerms(), loadAnalysis()]);
}

async function deleteTerm(id) {
  if (!confirm('Confirm delete?')) return;
  await request(`/terms/${id}`, { method: 'DELETE' });
  await loadTerms();
}

async function startPractice() {
  practiceResult.value = null;
  Object.keys(answers).forEach((key) => delete answers[key]);
  const data = await request('/practice/start', {
    method: 'POST',
    body: {
      ...practiceConfig,
      categoryId: practiceConfig.categoryId || null,
      difficulty: practiceConfig.difficulty || null
    }
  });
  questions.value = data.questions;
}

async function submitPractice() {
  const payload = questions.value.map((question, index) => ({
    ...question,
    userAnswer: answers[index] || ''
  }));
  practiceResult.value = await request('/practice/submit', {
    method: 'POST',
    body: {
      type: practiceConfig.type,

```

```

    categoryId: practiceConfig.categoryId || null,
    difficulty: practiceConfig.difficulty || null,
    answers: payload
  }
});
await Promise.all([loadAnalysis(), loadWrongQuestions()]);
}

onMounted(async () => {
  if (isLoggedIn.value) {
    try {
      await bootstrap();
    } catch (err) {
      error.value = err.message;
    }
  }
});
</script>

<template>
  <main v-if="!isLoggedIn" class="login-screen">
    <section class="login-panel">
      <div>
        <p class="eyebrow">Railway Terminology</p>
        <h1>{{ t.app }}</h1>
      </div>
      <div class="lang-row">
        <button v-for="item in ['zh','en','vi']" :key="item" :class="{ active: lang === item }"
@click="setLanguage(item)">
          {{ item.toUpperCase() }}
        </button>
      </div>
      <form class="form-stack" @submit.prevent="login">
        <label>{{ t.username }}<input v-model="loginForm.username" autocomplete="username" /></label>
        <label>{{ t.password }}<input v-model="loginForm.password" type="password" autocomplete="current-
password" /></label>
        <p v-if="error" class="error">{{ error }}</p>
        <button class="primary" :disabled="loading">{{ t.login }}</button>
      </form>
      <p class="hint">admin/** . teacher/** . student/**</p>
    </section>
  </main>

  <div v-else class="app-shell">
    <aside class="sidebar">
      <h2>{{ t.app }}</h2>
      <nav>
        <button :class="{ active: page === 'dashboard' }" @click="page = 'dashboard'">{{ t.dashboard }}
</button>
        <button :class="{ active: page === 'terms' }" @click="page = 'terms'">{{ t.terms }}</button>
        <button :class="{ active: page === 'practice' }" @click="page = 'practice'">{{ t.practice }}
</button>
        <button :class="{ active: page === 'analysis' }" @click="page = 'analysis'">{{ t.analysis }}
</button>
        <button v-if="canManage" :class="{ active: page === 'system' }" @click="page = 'system'">{{
t.system }}</button>
      </nav>
    </aside>

    <section class="workspace">
      <header class="topbar">
        <div>
          <strong>{{ user.realName || user.real_name }}</strong>
          <span>{{ user.roleCode || user.role_code }}</span>
        </div>
        <div class="toolbar">
          <select :value="lang" @change="setLanguage($event.target.value)">
            <option value="zh">中文</option>
            <option value="en">English</option>
            <option value="vi">Tiếng Việt</option>

```

```

</select>
<button @click="logout">{{ t.logout }}</button>
</div>
</header>

<section v-if="page === 'dashboard'" class="page">
  <div class="section-title">
    <h1>{{ t.dashboard }}</h1>
    <p>Chinese · Vietnamese · English railway terminology learning</p>
  </div>
  <div class="stats-grid">
    <article v-for="card in statsCards" :key="card.label">
      <span>{{ card.label }}</span>
      <strong>{{ card.value }}</strong>
    </article>
  </div>
  <div class="two-column">
    <section class="panel">
      <h3>{{ t.categories }}</h3>
      <div v-for="cat in adminAnalysis?.categories || []" :key="cat.name_zh" class="bar-row">
        <span>{{ cat.name_zh }}</span>
        <div><i :style="{ width: `${Math.min(100, cat.term_count * 12)}%` }"></i></div>
        <b>{{ cat.term_count }}</b>
      </div>
    </section>
    <section class="panel">
      <h3>Recent Terms</h3>
      <ul class="term-list">
        <li v-for="term in terms.slice(0, 6)" :key="term.id">
          <strong>{{ term.term_zh }}</strong>
          <span>{{ term.term_vi }} / {{ term.term_en }}</span>
        </li>
      </ul>
    </section>
  </div>
</section>

<section v-if="page === 'terms'" class="page">
  <div class="section-title">
    <h1>{{ t.terms }}</h1>
    <button v-if="canManage" class="primary" @click="newTerm">{{ t.addTerm }}</button>
  </div>
  <div class="filters">
    <input v-model="filters.keyword" :placeholder="t.keyword" @input="resetTermSearch" />
    <select v-model="filters.categoryId" @change="resetTermSearch">
      <option value="">{{ t.categories }}</option>
      <option v-for="cat in categories" :key="cat.id" :value="cat.id">{{ cat.name_zh }}</option>
    </select>
    <select v-model="filters.difficulty" @change="resetTermSearch">
      <option value="">All</option>
      <option value="easy">Easy</option>
      <option value="medium">Medium</option>
      <option value="hard">Hard</option>
    </select>
  </div>
  <div class="table-wrap">
    <table>
      <thead><tr><th>中文</th><th>Tiếng Việt</th><th>English</th><th>Category</th><th>Level</th><th>
</th></tr></thead>
      <tbody>
        <tr v-for="term in terms" :key="term.id">
          <td><strong>{{ term.term_zh }}</strong><small>{{ term.pinyin }}</small></td>
          <td>{{ term.term_vi }}</td>
          <td>{{ term.term_en }}</td>
          <td>{{ term.category_zh }}</td>
          <td><span class="tag">{{ term.difficulty }}</span></td>
          <td class="actions" v-if="canManage">
            <button @click="editTerm(term)">Edit</button>
            <button @click="deleteTerm(term.id)">Delete</button>
          </td>
        </tr>
      </tbody>
    </table>
  </div>

```

```

        </tbody>
    </table>
</div>
<div class="pagination">
    <button :disabled="termPager.page <= 1" @click="changePage(termPager, termPager.page - 1,
loadTerms)">Prev</button>
    <span>{{ termPager.page }} / {{ pageCount(termPager) }} · {{ termPager.total }}</span>
    <button :disabled="termPager.page >= pageCount(termPager)" @click="changePage(termPager,
termPager.page + 1, loadTerms)">Next</button>
</div>
</section>

<section v-if="page === 'practice'" class="page">
    <div class="section-title"><h1>{{ t.practice }}</h1></div>
    <div class="panel practice-config">
        <select v-model="practiceConfig.type">
            <option value="choice">Choice</option>
            <option value="judge">Judge</option>
            <option value="fill">Fill</option>
            <option value="translate">Translate</option>
        </select>
        <select v-model="practiceConfig.categoryId">
            <option value="">All Categories</option>
            <option v-for="cat in categories" :key="cat.id" :value="cat.id">{{ cat.name_zh }}</option>
        </select>
        <select v-model="practiceConfig.difficulty">
            <option value="">All Levels</option>
            <option value="easy">Easy</option>
            <option value="medium">Medium</option>
            <option value="hard">Hard</option>
        </select>
        <input v-model.number="practiceConfig.count" type="number" min="1" max="20" />
        <button class="primary" @click="startPractice">{{ t.start }}</button>
    </div>
    <div class="question-stack">
        <article v-for="(question, index) in questions" :key="`${question.termId}-${index}`"
class="question">
            <h3>{{ index + 1 }}. {{ question.questionText }}</h3>
            <div v-if="question.options.length" class="options">
                <label v-for="option in question.options" :key="option">
                    <input v-model="answers[index]" type="radio" :name="`q-${index}`" :value="option" />
                    {{ option === 'true' ? '正确 / True' : option === 'false' ? '错误 / False' : option }}
                </label>
            </div>
            <input v-else v-model="answers[index]" placeholder="Answer" />
            <p v-if="practiceResult" :class="practiceResult.answers[index]?.isCorrect ? 'ok' : 'error'">
                {{ practiceResult.answers[index]?.isCorrect ? 'Correct' : `Wrong: ${question.correctAnswer}`
}}
            </p>
        </article>
    </div>
    <button v-if="questions.length && !practiceResult" class="primary submit" @click="submitPractice">
{{ t.submit }}</button>
    <section v-if="practiceResult" class="result-panel">
        <strong>{{ practiceResult.score }}</strong>
        <span>{{ practiceResult.correct }} / {{ practiceResult.total }}</span>
    </section>
</section>

<section v-if="page === 'analysis'" class="page">
    <div class="section-title"><h1>{{ t.analysis }}</h1></div>
    <div class="stats-grid">
        <article v-for="card in statsCards" :key="card.label"><span>{{ card.label }}</span><strong>{{
card.value }}</strong></article>
    </div>
    <div class="two-column">
        <section class="panel">
            <h3>Trend</h3>
            <div v-for="item in studentAnalysis?.trend || []" :key="item.date" class="bar-row">

```

```

        <span>{{ item.date }}</span>
      </div>
    </div>
  </section>
  <section class="panel">
    <h3>{{ t.wrongBook }}</h3>
    <ul class="term-list">
      <li v-for="item in wrongQuestions" :key="item.id">
        <strong>{{ item.term_zh }} / {{ item.term_vi }}</strong>
        <span>{{ item.category_zh }} · {{ item.wrong_count }}</span>
      </li>
    </ul>
  </section>
</div>
</section>

<section v-if="page === 'system' && canManage" class="page">
  <div class="section-title"><h1>{{ t.system }}</h1></div>
  <div class="two-column">
    <section class="panel">
      <h3>{{ t.users }}</h3>
      <table><tbody><tr v-for="item in users" :key="item.id"><td>{{ item.username }}</td><td>{{
item.real_name }}</td><td>{{ item.role_name }}</td><td>{{ item.status }}</td></tr></tbody></table>
      <div class="pagination compact">
        <button :disabled="userPager.page <= 1" @click="changePage(userPager, userPager.page - 1,
loadUsers)">Prev</button>
        <span>{{ userPager.page }} / {{ pageCount(userPager) }} · {{ userPager.total }}</span>
        <button :disabled="userPager.page >= pageCount(userPager)" @click="changePage(userPager,
userPager.page + 1, loadUsers)">Next</button>
      </div>
    </section>
    <section class="panel">
      <h3>{{ t.logs }}</h3>
      <ul class="log-list">
        <li v-for="log in loginLogs.slice(0, 12)" :key="log.id">
          <span>{{ log.username }}</span>
          <b :class="log.success ? 'ok' : 'error'">{{ log.success ? 'OK' : 'FAIL' }}</b>
          <small>{{ log.created_at }}</small>
        </li>
      </ul>
      <div class="pagination compact">
        <button :disabled="logPager.page <= 1" @click="changePage(logPager, logPager.page - 1,
loadLogs)">Prev</button>
        <span>{{ logPager.page }} / {{ pageCount(logPager) }} · {{ logPager.total }}</span>
        <button :disabled="logPager.page >= pageCount(logPager)" @click="changePage(logPager,
logPager.page + 1, loadLogs)">Next</button>
      </div>
    </section>
  </div>
</section>
</section>

<div v-if="showTermForm" class="modal-backdrop">
  <form class="modal" @submit.prevent="saveTerm">
    <h2>{{ editingId ? 'Edit Term' : t.addTerm }}</h2>
    <div class="form-grid">
      <label>中文<input v-model="termForm.term_zh" required /></label>
      <label>Tiếng Việt<input v-model="termForm.term_vi" required /></label>
      <label>English<input v-model="termForm.term_en" required /></label>
      <label>Pinyin<input v-model="termForm.pinyin" /></label>
      <label>Category<select v-model="termForm.category_id" required><option v-for="cat in categories"
:key="cat.id" :value="cat.id">{{ cat.name_zh }}</option></select></label>
      <label>Difficulty<select v-model="termForm.difficulty"><option value="easy">easy</option><option
value="medium">medium</option><option value="hard">hard</option></select></label>
    </div>
    <label>中文释义<textarea v-model="termForm.definition_zh"></textarea></label>
    <label>越南语释义<textarea v-model="termForm.definition_vi"></textarea></label>
    <label>English Definition<textarea v-model="termForm.definition_en"></textarea></label>
    <div class="modal-actions">

```

```

    <button type="button" @click="showForm = false" {{ t.cancel }}></button>
    <button class="primary">{{ t.save }}</button>
  </div>
</form>
</div>
</div>
</template>

```

3.5 client/src/api/http.js

文件路径：client/src/api/http.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

const API_BASE = '/api';

export function getToken() {
  return localStorage.getItem('token');
}

export function setToken(token) {
  localStorage.setItem('token', token);
}

export function clearToken() {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
}

export async function request(path, options = {}) {
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
  const token = getToken();
  if (token) headers.Authorization = `Bearer ${token}`;
  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers,
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.message || 'Request failed');
  return data;
}

```

3.6 client/src/assets/styles.css

文件路径：client/src/assets/styles.css

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

:root {
  color: #172026;
  background: #f6f8fb;
  font-family: Inter, "Segoe UI", Arial, "Microsoft YaHei", sans-serif;
  font-size: 16px;
  letter-spacing: 0;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
}

```

```
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  border: 1px solid #cfd8e3;
  background: #ffffff;
  color: #172026;
  border-radius: 6px;
  padding: 9px 13px;
  cursor: pointer;
}

button.primary,
.primary {
  border-color: #0f766e;
  background: #0f766e;
  color: #ffffff;
}

button.active {
  border-color: #0f766e;
  background: #e7f6f3;
  color: #0f766e;
}

input,
select,
textarea {
  width: 100%;
  border: 1px solid #cfd8e3;
  border-radius: 6px;
  background: #ffffff;
  padding: 10px 12px;
  color: #172026;
}

textarea {
  min-height: 74px;
  resize: vertical;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  padding: 13px 12px;
  border-bottom: 1px solid #e5eaf0;
  text-align: left;
  vertical-align: top;
}

th {
  color: #607080;
  font-size: 0.88rem;
  font-weight: 700;
  background: #f8fafc;
}

td small {
  display: block;
  color: #758292;
```

```
margin-top: 4px;
}

.login-screen {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 24px;
  background: linear-gradient(145deg, #eef7f5 0%, #f7f9fb 52%, #fff4e8 100%);
}

.login-panel {
  width: min(460px, 100%);
  background: #ffffff;
  border: 1px solid #e1e7ee;
  border-radius: 8px;
  padding: 34px;
  box-shadow: 0 18px 60px rgba(30, 44, 58, 0.12);
}

.login-panel h1 {
  margin: 6px 0 22px;
  font-size: 1.8rem;
  line-height: 1.22;
}

.eyebrow {
  margin: 0;
  color: #0f766e;
  font-weight: 700;
  text-transform: uppercase;
  font-size: 0.78rem;
}

.lang-row,
.toolbar,
.filters,
.practice-config,
.modal-actions {
  display: flex;
  gap: 10px;
  align-items: center;
  flex-wrap: wrap;
}

.lang-row {
  margin-bottom: 22px;
}

.form-stack {
  display: grid;
  gap: 14px;
}

label {
  display: grid;
  gap: 7px;
  color: #4b5b68;
  font-size: 0.92rem;
  font-weight: 650;
}

.hint {
  margin: 18px 0 0;
  color: #758292;
  font-size: 0.88rem;
}

.error {
  color: #b42318;
```

```
}

.ok {
  color: #0f766e;
}

.app-shell {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 260px minmax(0, 1fr);
}

.sidebar {
  background: #172026;
  color: #ffffff;
  padding: 24px 18px;
}

.sidebar h2 {
  margin: 0 0 28px;
  font-size: 1.08rem;
  line-height: 1.35;
}

.sidebar nav {
  display: grid;
  gap: 8px;
}

.sidebar button {
  width: 100%;
  text-align: left;
  color: #d8e1e8;
  border-color: transparent;
  background: transparent;
}

.sidebar button.active,
.sidebar button:hover {
  background: #26343d;
  color: #ffffff;
}

.workspace {
  min-width: 0;
}

.topbar {
  height: 72px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 26px;
  background: #ffffff;
  border-bottom: 1px solid #e1e7ee;
}

.topbar span {
  display: block;
  color: #758292;
  font-size: 0.86rem;
  margin-top: 3px;
}

.topbar select {
  width: 138px;
}

.page {
  padding: 26px;
```

```
}

.section-title {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 20px;
  margin-bottom: 22px;
}

.section-title h1 {
  margin: 0;
  font-size: 1.7rem;
}

.section-title p {
  margin: 6px 0 0;
  color: #607080;
}

.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 18px;
}

.stats-grid article,
.panel,
.question,
.result-panel {
  background: #ffffff;
  border: 1px solid #e1e7ee;
  border-radius: 8px;
}

.stats-grid article {
  padding: 18px;
}

.stats-grid span {
  color: #607080;
}

.stats-grid strong {
  display: block;
  margin-top: 10px;
  font-size: 2rem;
}

.two-column {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  gap: 18px;
}

.panel {
  padding: 18px;
}

.panel h3 {
  margin: 0 0 16px;
}

.bar-row {
  display: grid;
  grid-template-columns: 130px 1fr 44px;
  gap: 12px;
  align-items: center;
  margin: 12px 0;
```

```
}

.bar-row div {
  height: 10px;
  overflow: hidden;
  background: #e7edf2;
  border-radius: 999px;
}

.bar-row i {
  display: block;
  height: 100%;
  background: #0f766e;
}

.term-list,
.log-list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 11px;
}

.term-list li,
.log-list li {
  display: grid;
  gap: 4px;
  padding-bottom: 10px;
  border-bottom: 1px solid #eef2f6;
}

.term-list span,
.log-list small {
  color: #607080;
}

.filters {
  margin-bottom: 16px;
}

.filters input {
  max-width: 280px;
}

.filters select {
  max-width: 190px;
}

.table-wrap {
  overflow: auto;
  background: #ffffff;
  border: 1px solid #ele7ee;
  border-radius: 8px;
}

.tag {
  display: inline-flex;
  padding: 4px 8px;
  background: #f0f7f5;
  color: #0f766e;
  border-radius: 999px;
  font-size: 0.82rem;
  font-weight: 700;
}

.actions {
  white-space: nowrap;
}
```

```
.actions button {
  margin-right: 8px;
}

.pagination {
  display: flex;
  justify-content: flex-end;
  align-items: center;
  gap: 12px;
  margin-top: 14px;
  color: #607080;
}

.pagination.compact {
  justify-content: flex-start;
}

.pagination button:disabled {
  cursor: not-allowed;
  opacity: 0.45;
}

.practice-config {
  margin-bottom: 18px;
}

.practice-config select {
  width: 180px;
}

.practice-config input {
  width: 90px;
}

.question-stack {
  display: grid;
  gap: 14px;
}

.question {
  padding: 18px;
}

.question h3 {
  margin: 0 0 14px;
  font-size: 1rem;
}

.options {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
}

.options label {
  display: flex;
  align-items: center;
  gap: 9px;
  min-height: 44px;
  padding: 9px 12px;
  border: 1px solid #d7e0e8;
  border-radius: 6px;
  font-weight: 500;
}

.options input {
  width: auto;
}

.submit {
```

```
margin-top: 16px;
}

.result-panel {
  margin-top: 16px;
  padding: 20px;
  display: flex;
  align-items: baseline;
  gap: 12px;
}

.result-panel strong {
  font-size: 2.4rem;
  color: #0f766e;
}

.modal-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(23, 32, 38, 0.52);
  display: grid;
  place-items: center;
  padding: 20px;
}

.modal {
  width: min(780px, 100%);
  max-height: 90vh;
  overflow: auto;
  background: #ffffff;
  border-radius: 8px;
  padding: 24px;
  display: grid;
  gap: 14px;
}

.modal h2 {
  margin: 0;
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

@media (max-width: 900px) {
  .app-shell {
    grid-template-columns: 1fr;
  }

  .sidebar {
    position: sticky;
    top: 0;
    z-index: 2;
    padding: 14px;
  }

  .sidebar h2 {
    margin-bottom: 12px;
  }

  .sidebar nav {
    display: flex;
    overflow-x: auto;
  }

  .sidebar button {
    min-width: 118px;
    text-align: center;
  }
}
```

```
}

.stats-grid,
.two-column,
.options,
.form-grid {
  grid-template-columns: 1fr;
}

.topbar {
  height: auto;
  padding: 14px;
  align-items: flex-start;
  gap: 12px;
}

.page {
  padding: 16px;
}

.section-title {
  display: grid;
}

.bar-row {
  grid-template-columns: 100px 1fr 38px;
}
}
```

3.7 client/src/locales/messages.js

文件路径：client/src/locales/messages.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
export const messages = {
  zh: {
    app: '汉越铁道术语学习软件',
    login: '登录',
    logout: '退出',
    username: '用户名',
    password: '密码',
    dashboard: '学习看板',
    terms: '术语管理',
    practice: '练习平台',
    analysis: '数据分析',
    system: '系统管理',
    logs: '登录日志',
    categories: '分类',
    keyword: '关键词',
    addTerm: '新增术语',
    save: '保存',
    cancel: '取消',
    start: '开始练习',
    submit: '提交答案',
    wrongBook: '错题本',
    users: '用户管理',
    roles: '角色管理',
    totalTerms: '术语总数',
    avgScore: '平均得分',
    practiceCount: '练习次数',
    correctRate: '正确率'
  },
  en: {
    app: 'Chinese-Vietnamese Railway Term Learning',
    login: 'Login',
    logout: 'Logout',
    username: 'Username',
```

```
password: 'Password',
dashboard: 'Dashboard',
terms: 'Terms',
practice: 'Practice',
analysis: 'Analytics',
system: 'System',
logs: 'Login Logs',
categories: 'Categories',
keyword: 'Keyword',
addTerm: 'Add Term',
save: 'Save',
cancel: 'Cancel',
start: 'Start Practice',
submit: 'Submit',
wrongBook: 'Wrong Book',
users: 'Users',
roles: 'Roles',
totalTerms: 'Terms',
avgScore: 'Average Score',
practiceCount: 'Practice Count',
correctRate: 'Accuracy'
},
vi: {
  app: 'Phần mềm học thuật ngữ đường sắt Trung-Việt',
  login: 'Đăng nhập',
  logout: 'Đăng xuất',
  username: 'Tên người dùng',
  password: 'Mật khẩu',
  dashboard: 'Bảng học tập',
  terms: 'Quản lý thuật ngữ',
  practice: 'Luyện tập',
  analysis: 'Phân tích dữ liệu',
  system: 'Quản trị hệ thống',
  logs: 'Nhật ký đăng nhập',
  categories: 'Danh mục',
  keyword: 'Từ khóa',
  addTerm: 'Thêm thuật ngữ',
  save: 'Lưu',
  cancel: 'Hủy',
  start: 'Bắt đầu luyện tập',
  submit: 'Nộp bài',
  wrongBook: 'Số câu sai',
  users: 'Quản lý người dùng',
  roles: 'Vai trò',
  totalTerms: 'Tổng thuật ngữ',
  avgScore: 'Điểm trung bình',
  practiceCount: 'Số lần luyện tập',
  correctRate: 'Tỷ lệ đúng'
}
};
```

3.8 client/src/main.js

文件路径：client/src/main.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
import { createApp } from 'vue';
import App from './App.vue';
import './assets/styles.css';

createApp(App).mount('#app');
```

3.9 client/vite.config.js

文件路径：client/vite.config.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

面向越南留学生的铁道专业汉越术语翻译学习软件V1.0源代码

```
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
    proxy: {
      '/api': 'http://***:3000'
    }
  }
});
```

3.10 package.json

文件路径：package.json

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
{
  "name": "chinese-vietnamese-railway-term-learning",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "面向越南留学生的铁道专业汉越术语翻译学习软件 V1.0",
  "scripts": {
    "dev": "concurrently \"npm run dev --workspace server\" \"npm run dev --workspace client\"",
    "build": "npm run build --workspace client",
    "start": "npm run start --workspace server",
    "install:all": "npm install --workspaces"
  },
  "workspaces": [
    "client",
    "server"
  ],
  "devDependencies": {
    "concurrently": "^9.1.2"
  }
}
```

3.11 server/.env.example

文件路径：server/.env.example

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
PORT=3000
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=railway_term_learning
JWT_SECRET=***
JWT_EXPIRES_IN=8h
```

3.12 server/package.json

文件路径：server/package.json

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
{
  "name": "railway-term-learning-server",
  "version": "1.0.0",
```

```

    "private": true,
    "type": "module",
    "scripts": {
      "dev": "nodemon src/index.js",
      "start": "node src/index.js",
      "seed:test-data": "node scripts/seed-test-data.js",
      "test:smoke": "node scripts/smoke-test.js"
    },
    "dependencies": {
      "bcryptjs": "^2.4.3",
      "cors": "^2.8.5",
      "dotenv": "^16.4.7",
      "express": "^4.21.2",
      "helmet": "^8.0.0",
      "jsonwebtoken": "^9.0.2",
      "mysql2": "^3.12.0",
      "morgan": "^1.10.0",
      "nanoid": "^5.0.9",
      "zod": "^3.24.1"
    },
    "devDependencies": {
      "nodemon": "^3.1.9"
    }
  }
}

```

3.13 server/scripts/seed-test-data.js

文件路径：server/scripts/seed-test-data.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import bcrypt from 'bcryptjs';
import { initializeDatabase } from '../src/db/init.js';
import { getPool, query } from '../src/db/pool.js';

const MIN_ROWS = 100;

const zhTerms = ['铁路', '车站', '钢轨', '道岔', '机车', '客车', '信号机', '闭塞', '调度命令', '列车运行图'];
const viTerms = ['đường sắt', 'nhà ga', 'ray thép', 'ghì đường sắt', 'đầu máy', 'toa khách', 'tín hiệu', 'đóng đường', 'mệnh lệnh điều độ', 'biểu đồ chạy tàu'];
const enTerms = ['railway', 'station', 'rail', 'turnout', 'locomotive', 'coach', 'signal', 'block system', 'dispatching order', 'train diagram'];
const modules = ['auth', 'terms', 'practice', 'analysis', 'system'];
const actions = ['create', 'update', 'delete', 'query', 'export'];

function pad(num) {
  return String(num).padStart(3, '0');
}

async function count(table) {
  const rows = await query(`SELECT COUNT(*) AS count FROM ${table}`);
  return Number(rows[0].count);
}

async function ensureRoles() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT IGNORE INTO sys_role (code, name_zh, name_en, name_vi)
      VALUES (:code, :zh, :en, :vi)`,
      {
        code: `role_${pad(i)}`,
        zh: `测试角色${pad(i)}`,
        en: `Test Role ${pad(i)}`,
        vi: `Vai trò kiểm thử${pad(i)}`
      }
    );
  }
}

```

```

async function ensureUsers() {
  const hash = await bcrypt.hash('test123456', 10);
  const roles = await query('SELECT id FROM sys_role ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT IGNORE INTO sys_user (username, password_hash, real_name, email, role_id, status)
      VALUES (:username, :hash, :realName, :email, :roleId, 'enabled')`,
      {
        username: `test_user_${pad(i)}`,
        hash,
        realName: `测试用户${pad(i)}`,
        email: `test_user_${pad(i)}@example.com`,
        roleId: roles[(i - 1) % roles.length].id
      }
    );
  }
}

async function ensureCategories() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT INTO term_category (name_zh, name_en, name_vi, sort_order, status)
      SELECT :zh, :en, :vi, :sortOrder, 'enabled'
      WHERE NOT EXISTS (SELECT 1 FROM term_category WHERE name_zh = :zh)`,
      {
        zh: `测试分类${pad(i)}`,
        en: `Test Category ${pad(i)}`,
        vi: `Danh mục kiểm thử${pad(i)}`,
        sortOrder: 1000 + i
      }
    );
  }
}

async function ensureTerms() {
  const categories = await query('SELECT id FROM term_category ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    const idx = (i - 1) % zhTerms.length;
    await query(
      `INSERT INTO railway_term
      (category_id, term_zh, term_en, term_vi, pinyin, difficulty, definition_zh, definition_en,
      definition_vi,
      example_zh, example_en, example_vi, status, created_by)
      SELECT :categoryId, :zh, :en, :vi, :pinyin, :difficulty, :defZh, :defEn, :defVi, :exZh, :exEn,
      :exVi, 'enabled', 1
      WHERE NOT EXISTS (SELECT 1 FROM railway_term WHERE term_zh = :zh)`,
      {
        categoryId: categories[(i - 1) % categories.length].id,
        zh: `${zhTerms[idx]}测试${pad(i)}`,
        en: `${enTerms[idx]} test ${pad(i)}`,
        vi: `${viTerms[idx]} kiểm thử${pad(i)}`,
        pinyin: `pin yin ${pad(i)}`,
        difficulty: ['easy', 'medium', 'hard'][(i - 1) % 3],
        defZh: `测试术语${pad(i)}的中文释义。`,
        defEn: `English definition for test term ${pad(i)}.`,
        defVi: `Định nghĩa tiếng Việt cho thuật ngữ kiểm thử${pad(i)}.`,
        exZh: `这是测试术语${pad(i)}的示例句。`,
        exEn: `This is an example sentence for test term ${pad(i)}.`,
        exVi: `Đây là câu ví dụ cho thuật ngữ kiểm thử${pad(i)}.`
      }
    );
  }
}

async function ensureLoginLogs() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT INTO sys_login_log (username, ip, user_agent, success, message)
      SELECT :username, :ip, 'seed-test-agent', :success, :message`
    );
  }
}

```

```

    {
      username: `test_user_${pad(i)}`,
      ip: `*** % 200) + 1}`,
      success: i % 7 === 0 ? 0 : 1,
      message: i % 7 === 0 ? '测试登录失败' : '测试登录成功',
      minRows: MIN_ROWS
    }
  );
}
}

async function ensureOperationLogs() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT INTO sys_operation_log (user_id, module, action, detail)
      SELECT :userId, :module, :action, :detail
      WHERE (SELECT COUNT(*) FROM sys_operation_log) < :minRows`,
      {
        userId: ((i - 1) % MIN_ROWS) + 1,
        module: modules[(i - 1) % modules.length],
        action: actions[(i - 1) % actions.length],
        detail: `测试操作日志${pad(i)}`,
        minRows: MIN_ROWS
      }
    );
  }
}

async function ensurePracticeRecords() {
  const categories = await query('SELECT id FROM term_category ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    const total = 5 + (i % 6);
    const correct = i % (total + 1);
    await query(
      `INSERT INTO practice_record (user_id, practice_type, category_id, difficulty, total_count,
correct_count, score, duration_seconds)
      SELECT :userId, :type, :categoryId, :difficulty, :total, :correct, :score, :duration
      WHERE (SELECT COUNT(*) FROM practice_record) < :minRows`,
      {
        userId: ((i - 1) % MIN_ROWS) + 1,
        type: ['choice', 'judge', 'fill', 'translate'][(i - 1) % 4],
        categoryId: categories[(i - 1) % categories.length].id,
        difficulty: ['easy', 'medium', 'hard'][(i - 1) % 3],
        total,
        correct,
        score: Number(((correct / total) * 100).toFixed(2)),
        duration: 30 + i,
        minRows: MIN_ROWS
      }
    );
  }
}

async function ensurePracticeAnswers() {
  const records = await query('SELECT id FROM practice_record ORDER BY id LIMIT 100');
  const terms = await query('SELECT id, term_zh, term_vi FROM railway_term ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    const term = terms[(i - 1) % terms.length];
    const isCorrect = i % 4 !== 0;
    await query(
      `INSERT INTO practice_answer (record_id, term_id, question_type, question_text, user_answer,
correct_answer, is_correct)
      SELECT :recordId, :termId, :type, :question, :userAnswer, :correctAnswer, :isCorrect
      WHERE (SELECT COUNT(*) FROM practice_answer) < :minRows`,
      {
        recordId: records[(i - 1) % records.length].id,
        termId: term.id,
        type: ['choice', 'judge', 'fill', 'translate'][(i - 1) % 4],
        question: `测试题目${pad(i)}: ${term.term_zh}`,

```

```

        userAnswer: isCorrect ? term.term_vi : '错误答案',
        correctAnswer: term.term_vi,
        isCorrect: isCorrect ? 1 : 0,
        minRows: MIN_ROWS
    }
});
}
}

async function ensureWrongQuestions() {
    const terms = await query('SELECT id FROM railway_term ORDER BY id LIMIT 100');
    for (let i = 1; i <= MIN_ROWS; i += 1) {
        await query(
            `INSERT IGNORE INTO wrong_question (user_id, term_id, wrong_count)
            VALUES (:userId, :termId, :wrongCount)\`,
            {
                userId: ((i - 1) % MIN_ROWS) + 1,
                termId: terms[(i - 1) % terms.length].id,
                wrongCount: (i % 5) + 1
            }
        );
    }
}

async function ensureFavorites() {
    const terms = await query('SELECT id FROM railway_term ORDER BY id LIMIT 100');
    for (let i = 1; i <= MIN_ROWS; i += 1) {
        await query(
            `INSERT IGNORE INTO favorite_term (user_id, term_id)
            VALUES (:userId, :termId)\`,
            {
                userId: ((i - 1) % MIN_ROWS) + 1,
                termId: terms[(MIN_ROWS - i) % terms.length].id
            }
        );
    }
}

async function main() {
    await initializeDatabase();
    await ensureRoles();
    await ensureUsers();
    await ensureCategories();
    await ensureTerms();
    await ensureLoginLogs();
    await ensureOperationLogs();
    await ensurePracticeRecords();
    await ensurePracticeAnswers();
    await ensureWrongQuestions();
    await ensureFavorites();

    const tables = [
        'sys_role',
        'sys_user',
        'sys_login_log',
        'sys_operation_log',
        'term_category',
        'railway_term',
        'practice_record',
        'practice_answer',
        'wrong_question',
        'favorite_term'
    ];
    const rows = [];
    for (const table of tables) rows.push({ table, count: await count(table) });
    console.table(rows);
    const failures = rows.filter((row) => row.count < MIN_ROWS);
    await getPool().end();
    if (failures.length) {
        console.error('Tables below minimum rows:', failures);
    }
}

```

```

    process.exit(1);
  }
}

main().catch(async (error) => {
  console.error(error);
  await getPool().end();
  process.exit(1);
});

```

3.14 server/scripts/smoke-test.js

文件路径：server/scripts/smoke-test.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

const BASE_URL = process.env.BASE_URL || 'http://***:3000/api';

async function http(path, options = {}, token = '') {
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  if (token) headers.Authorization = `Bearer ${token}`;
  const response = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers,
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const text = await response.text();
  const data = text ? JSON.parse(text) : {};
  if (!response.ok) {
    throw new Error(`${options.method || 'GET'} ${path} failed: ${response.status} ${data.message || text}`);
  }
  return data;
}

function assert(condition, message) {
  if (!condition) throw new Error(message);
}

function assertPage(data, page, pageSize, minTotal, name) {
  assert(Array.isArray(data.rows), `${name} rows missing`);
  assert(data.page === page, `${name} page mismatch`);
  assert(data.pageSize === pageSize, `${name} pageSize mismatch`);
  assert(data.total >= minTotal, `${name} total below ${minTotal}`);
  assert(data.rows.length <= pageSize, `${name} returned too many rows`);
}

async function main() {
  const health = await http('/health');
  assert(health.status === 'ok', 'health check failed');

  const login = await http('/auth/login', {
    method: 'POST',
    body: { username: 'admin', password: '***' }
  });
  assert(login.token, 'login token missing');
  const token = login.token;

  const profile = await http('/auth/profile', {}, token);
  assert(profile.username === 'admin', 'profile mismatch');

  const categoryPage = await http('/terms/categories?page=2&pageSize=10', {}, token);
  assertPage(categoryPage, 2, 10, 100, 'categories pagination');
  const categories = await http('/terms/categories', {}, token);
  assert(categories.length >= 100, 'category rows below 100');

  const category = await http('/terms/categories', {
    method: 'POST',

```

```

    body: {
      name_zh: `接口测试分类${Date.now()}`,
      name_en: 'API Test Category',
      name_vi: 'Danh mục kiểm thử'API',
      sort_order: 9999,
      status: 'enabled'
    }
  }, token);
await http(`/terms/categories/${category.id}`, {
  method: 'PUT',
  body: {
    name_zh: `接口测试分类更新${Date.now()}`,
    name_en: 'API Test Category Updated',
    name_vi: 'Danh mục kiểm thử'API cập nhật',
    sort_order: 9999,
    status: 'enabled'
  }
}, token);

const terms = await http('/terms?page=2&pageSize=10', {}, token);
assertPage(terms, 2, 10, 100, 'terms pagination');

const term = await http('/terms', {
  method: 'POST',
  body: {
    category_id: categories[0].id,
    term_zh: `接口测试术语${Date.now()}`,
    term_en: 'api test term',
    term_vi: 'thuật ngữ kiểm thử'api',
    pinyin: 'jie kou ce shi shu yu',
    difficulty: 'easy',
    definition_zh: '接口测试中文释义',
    definition_en: 'API test English definition',
    definition_vi: 'Định nghĩa tiếng Việt kiểm thử'API',
    example_zh: '接口测试示例句',
    example_en: 'API test example',
    example_vi: 'Ví dụ kiểm thử'API',
    status: 'enabled'
  }
}, token);
await http(`/terms/${term.id}`, {
  method: 'PUT',
  body: {
    category_id: categories[0].id,
    term_zh: `接口测试术语更新${Date.now()}`,
    term_en: 'api test term updated',
    term_vi: 'thuật ngữ kiểm thử'api cập nhật',
    pinyin: 'jie kou ce shi shu yu',
    difficulty: 'medium',
    definition_zh: '接口测试中文释义更新',
    definition_en: 'API test English definition updated',
    definition_vi: 'Định nghĩa tiếng Việt kiểm thử'API cập nhật',
    example_zh: '接口测试示例句更新',
    example_en: 'API test example updated',
    example_vi: 'Ví dụ kiểm thử'API cập nhật',
    status: 'enabled'
  }
}, token);
await http(`/terms/${term.id}/favorite`, { method: 'POST' }, token);
await http(`/terms/${term.id}`, { method: 'DELETE' }, token);
await http(`/terms/categories/${category.id}`, { method: 'DELETE' }, token);

const practice = await http('/practice/start', {
  method: 'POST',
  body: { type: 'choice', count: 5, categoryId: null, difficulty: null }
}, token);
assert(practice.questions.length > 0, 'practice question generation failed');
const result = await http('/practice/submit', {
  method: 'POST',
  body: {

```

```

    type: 'choice',
    categoryId: null,
    difficulty: null,
    answers: practice.questions.map((question) => ({
      ...question,
      userAnswer: question.correctAnswer
    })))
  }
}, token);
assert(Number(result.score) === 100, 'practice scoring failed');

const records = await http('/practice/records?page=1&pageSize=10', {}, token);
assertPage(records, 1, 10, 1, 'practice records pagination');
const wrongPage = await http('/practice/wrong-questions?page=1&pageSize=10', {}, token);
assertPage(wrongPage, 1, 10, 1, 'wrong questions pagination');

const studentAnalysis = await http('/analysis/student', {}, token);
assert(studentAnalysis.summary, 'student analysis missing');
const adminAnalysis = await http('/analysis/admin', {}, token);
assert(adminAnalysis.summary, 'admin analysis missing');

const users = await http('/system/users?page=2&pageSize=10', {}, token);
assertPage(users, 2, 10, 100, 'users pagination');
const roles = await http('/system/roles?page=2&pageSize=10', {}, token);
assertPage(roles, 2, 10, 100, 'roles pagination');
const loginLogs = await http('/system/logs/login?page=2&pageSize=10', {}, token);
assertPage(loginLogs, 2, 10, 100, 'login logs pagination');
const operationLogs = await http('/system/logs/operation?page=2&pageSize=10', {}, token);
assertPage(operationLogs, 2, 10, 100, 'operation logs pagination');

console.log('Smoke test passed: auth, paginated terms/categories/users/roles/logs/practice, analysis.');
```

```

}

main().catch((error) => {
  console.error(error.message);
  process.exit(1);
});

```

3.15 server/src/config/env.js

文件路径：server/src/config/env.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import dotenv from 'dotenv';

dotenv.config();

export const env = {
  port: Number(process.env.PORT || 3000),
  db: {
    host: process.env.DB_HOST || '***',
    port: Number(process.env.DB_PORT || 3306),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '***',
    database: process.env.DB_NAME || 'railway_term_learning'
  },
  jwtSecret: process.env.JWT_SECRET || '***',
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '8h'
};

```

3.16 server/src/db/init.js

文件路径：server/src/db/init.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import bcrypt from 'bcryptjs';
import { ensureDatabase, getPool, query } from './pool.js';

const schema = [
  `CREATE TABLE IF NOT EXISTS sys_role (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(40) NOT NULL UNIQUE,
    name_zh VARCHAR(80) NOT NULL,
    name_en VARCHAR(80) NOT NULL,
    name_vi VARCHAR(80) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS sys_user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(120) NOT NULL,
    real_name VARCHAR(80) NOT NULL,
    email VARCHAR(120),
    role_id INT NOT NULL,
    status ENUM('enabled','disabled') DEFAULT 'enabled',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id) REFERENCES sys_role(id)
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS sys_login_log (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    ip VARCHAR(80),
    user_agent VARCHAR(255),
    success TINYINT(1) NOT NULL,
    message VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS sys_operation_log (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    module VARCHAR(60) NOT NULL,
    action VARCHAR(60) NOT NULL,
    detail VARCHAR(500),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS term_category (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name_zh VARCHAR(100) NOT NULL,
    name_en VARCHAR(100) NOT NULL,
    name_vi VARCHAR(100) NOT NULL,
    sort_order INT DEFAULT 0,
    status ENUM('enabled','disabled') DEFAULT 'enabled',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS railway_term (
    id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    term_zh VARCHAR(160) NOT NULL,
    term_en VARCHAR(160) NOT NULL,
    term_vi VARCHAR(160) NOT NULL,
    pinyin VARCHAR(200),
    difficulty ENUM('easy','medium','hard') DEFAULT 'easy',
    definition_zh TEXT,
    definition_en TEXT,
    definition_vi TEXT,
    example_zh TEXT,
    example_en TEXT,
    example_vi TEXT,
    status ENUM('enabled','disabled') DEFAULT 'enabled',
    created_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES term_category(id)
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`
];

```

];

];

used to carry passengers.', 'Toa xe đường sắt dùng để' chở hành khách.', '客车内部配有座椅。', 'Passenger

coaches are equipped with seats.', '面向越南留学生的铁路专业汉语术语翻译学习软件 V1.0 源代码

[4, '信号机', 'signal', 'tín hiệu đường sắt', 'xin hao ji', 'medium', '向司机显示行车条件的信号设备。', 'A device that displays movement authority to drivers.', 'Thiết bị hiển thị điều kiện chạy tàu cho lái tàu.', '司机必须确认信号机显示。', 'Drivers must confirm the signal aspect.', 'Lái tàu phải xác nhận hiển thị tín hiệu.'],

[4, '闭塞', 'block system', 'hệ thống đóng đường', 'bi se', 'hard', '保证区间内列车安全间隔的行车制度。', 'An operating system that ensures safe train separation.', 'Chế độ vận hành bảo đảm khoảng cách an toàn giữa các tàu.', '自动闭塞提高线路通过能力。', 'Automatic block systems improve line capacity.', 'Đóng đường tự động nâng cao năng lực thông qua tuyến.'],

[5, '调度命令', 'dispatching order', 'mệnh lệnh điều độ', 'diao du ming ling', 'hard', '调度员发布的行车组织指令。', 'An operation instruction issued by a dispatcher.', 'Chỉ lệnh tổ chức chạy tàu do điều độ viên ban hành.', '调度命令必须准确传达。', 'Dispatching orders must be communicated accurately.', 'Mệnh lệnh điều độ phải được truyền đạt chính xác.'],

[5, '列车运行图', 'train diagram', 'biểu đồ chạy tàu', 'lie che yun xing tu', 'hard', '规定列车运行时间和顺序的技术文件。', 'A technical document defining train times and sequence.', 'Tài liệu kỹ thuật quy định thời gian và thứ tự chạy tàu.', '运行图是调度工作的基础。', 'The train diagram is the basis of dispatching.', 'Biểu đồ chạy tàu là cơ sở của công tác điều độ.'],

];

```
export async function initializeDatabase() {
  await ensureDatabase();
  const pool = getPool();
  for (const sql of schema) await pool.query(sql);

  const roleCount = await query('SELECT COUNT(*) AS count FROM sys_role');
  if (roleCount[0].count === 0) {
    await query(`INSERT INTO sys_role (code, name_zh, name_en, name_vi) VALUES
      ('admin','系统管理员','Administrator','Quản trị viên'),
      ('teacher','教师','Teacher','Giáo viên'),
      ('student','学生','Student','Sinh viên')`);
  }

  const userCount = await query('SELECT COUNT(*) AS count FROM sys_user');
  if (userCount[0].count === 0) {
    const adminHash = await bcrypt.hash('***', 10);
    const teacherHash = await bcrypt.hash('***', 10);
    const studentHash = await bcrypt.hash('***', 10);
    await query(`INSERT INTO sys_user (username, password_hash, real_name, email, role_id) VALUES
      ('admin', :adminHash, '系统管理员', '***@***', 1),
      ('teacher', :teacherHash, '术语教师', '***@***', 2),
      ('student', :studentHash, '越南留学生', '***@***', 3)` , { adminHash, teacherHash, studentHash });
  }

  const categoryCount = await query('SELECT COUNT(*) AS count FROM term_category');
  if (categoryCount[0].count === 0) {
    for (const item of categories) {
      await query('INSERT INTO term_category (name_zh, name_en, name_vi, sort_order) VALUES (:zh, :en, :vi, :sort)', {
        zh: item[0], en: item[1], vi: item[2], sort: item[3]
      });
    }
  }

  const termCount = await query('SELECT COUNT(*) AS count FROM railway_term');
  if (termCount[0].count === 0) {
    for (const t of terms) {
      await query(`INSERT INTO railway_term
        (category_id, term_zh, term_en, term_vi, pinyin, difficulty, definition_zh, definition_en,
        definition_vi, example_zh, example_en, example_vi, created_by)
        VALUES (:categoryId, :zh, :en, :vi, :pinyin, :difficulty, :defZh, :defEn, :defVi, :exZh, :exEn,
        :exVi, 1)` , {
        categoryId: t[0], zh: t[1], en: t[2], vi: t[3], pinyin: t[4], difficulty: t[5],
        defZh: t[6], defEn: t[7], defVi: t[8], exZh: t[9], exEn: t[10], exVi: t[11]
      });
    }
  }
}
```

3.17 server/src/db/pool.js

面向越南留学生的铁道专业汉越术语翻译学习软件V1.0源代码

文件路径：server/src/db/pool.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
import mysql from 'mysql2/promise';
import { env } from '../config/env.js';

let pool;

export async function ensureDatabase() {
  const server = await mysql.createConnection({
    host: env.db.host,
    port: env.db.port,
    user: env.db.user,
    password: env.db.password,
    multipleStatements: true
  });
  await server.query(`CREATE DATABASE IF NOT EXISTS \`${env.db.database}\` DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci`);
  await server.end();
}

export function getPool() {
  if (!pool) {
    pool = mysql.createPool({
      ...env.db,
      waitForConnections: true,
      connectionLimit: 10,
      namedPlaceholders: true,
      timezone: '+08:00'
    });
  }
  return pool;
}

export async function query(sql, params = {}) {
  const [rows] = await getPool().execute(sql, params);
  return rows;
}
```

3.18 server/src/db/schema.sql

文件路径：server/src/db/schema.sql

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
CREATE TABLE IF NOT EXISTS sys_role (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(40) NOT NULL UNIQUE,
  name_zh VARCHAR(80) NOT NULL,
  name_en VARCHAR(80) NOT NULL,
  name_vi VARCHAR(80) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS sys_user (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(120) NOT NULL,
  real_name VARCHAR(80) NOT NULL,
  email VARCHAR(120),
  role_id INT NOT NULL,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```

FOREIGN KEY (role_id) REFERENCES sys_role(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS sys_login_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  ip VARCHAR(80),
  user_agent VARCHAR(255),
  success TINYINT(1) NOT NULL,
  message VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS sys_operation_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  module VARCHAR(60) NOT NULL,
  action VARCHAR(60) NOT NULL,
  detail VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS term_category (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name_zh VARCHAR(100) NOT NULL,
  name_en VARCHAR(100) NOT NULL,
  name_vi VARCHAR(100) NOT NULL,
  sort_order INT DEFAULT 0,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS railway_term (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  term_zh VARCHAR(160) NOT NULL,
  term_en VARCHAR(160) NOT NULL,
  term_vi VARCHAR(160) NOT NULL,
  pinyin VARCHAR(200),
  difficulty ENUM('easy','medium','hard') DEFAULT 'easy',
  definition_zh TEXT,
  definition_en TEXT,
  definition_vi TEXT,
  example_zh TEXT,
  example_en TEXT,
  example_vi TEXT,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_by INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES term_category(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS practice_record (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  practice_type VARCHAR(40) NOT NULL,
  category_id INT,
  difficulty VARCHAR(20),
  total_count INT NOT NULL,
  correct_count INT NOT NULL,
  score DECIMAL(5,2) NOT NULL,
  duration_seconds INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS practice_answer (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  record_id BIGINT NOT NULL,
  term_id INT NOT NULL,

```

```

question_type VARCHAR(40) NOT NULL,
question_text VARCHAR(255) NOT NULL,
user_answer VARCHAR(255),
correct_answer VARCHAR(255) NOT NULL,
is_correct TINYINT(1) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (record_id) REFERENCES practice_record(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE IF NOT EXISTS wrong_question (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  wrong_count INT DEFAULT 1,
  last_wrong_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_user_term (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE IF NOT EXISTS favorite_term (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_fav (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

3.19 server/src/index.js

文件路径：server/src/index.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import { env } from './config/env.js';
import { initializeDatabase } from './db/init.js';
import { authRouter } from './routes/auth.js';
import { termsRouter } from './routes/terms.js';
import { practiceRouter } from './routes/practice.js';
import { analysisRouter } from './routes/analysis.js';
import { systemRouter } from './routes/system.js';

const app = express();

app.use(helmet());
app.use(cors());
app.use(express.json({ limit: '2mb' }));
app.use(morgan('dev'));

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', name: 'railway-term-learning-server' });
});
app.use('/api/auth', authRouter);
app.use('/api/terms', termsRouter);
app.use('/api/practice', practiceRouter);
app.use('/api/analysis', analysisRouter);
app.use('/api/system', systemRouter);

app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ message: err.message || 'Internal Server Error' });
});

initializeDatabase()
  .then(() => {
    app.listen(env.port, '0.0.0.0', () => {

```

```

    });
  });
  .catch((error) => {
    console.error('Database initialization failed:', error.message);
    process.exit(1);
  });
});

```

3.20 server/src/middleware/auth.js

文件路径：server/src/middleware/auth.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { query } from '../db/pool.js';

export async function authRequired(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    if (!token) return res.status(401).json({ message: 'Unauthorized' });
    const payload = jwt.verify(token, env.jwtSecret);
    const users = await query(
      `SELECT u.id, u.username, u.real_name, u.email, u.status, r.code AS role_code, r.name_zh AS role_name
      FROM sys_user u JOIN sys_role r ON u.role_id = r.id WHERE u.id = :id`,
      { id: payload.id }
    );
    if (!users.length || users[0].status !== 'enabled') return res.status(401).json({ message: 'Unauthorized' });
    req.user = users[0];
    next();
  } catch (error) {
    res.status(401).json({ message: 'Unauthorized' });
  }
}

export function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role_code)) return res.status(403).json({ message: 'Forbidden' });
    next();
  };
}

```

3.21 server/src/routes/analysis.js

文件路径：server/src/routes/analysis.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import { Router } from 'express';
import { authRequired } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const analysisRouter = Router();
analysisRouter.use(authRequired);

analysisRouter.get('/student', asyncHandler(async (req, res) => {
  const userId = req.query.userId || req.user.id;
  const summary = await query(
    `SELECT COUNT(*) AS practice_count,
      COALESCE(SUM(total_count),0) AS answer_count,
      COALESCE(SUM(correct_count),0) AS correct_count,
      COALESCE(ROUND(AVG(score),2),0) AS avg_score
      FROM practice_record WHERE user_id = :userId`,
    { id: userId }
  );
  res.json(summary);
}));

```

```

    { userId }
  );
  const wrong = await query('SELECT COUNT(*) AS wrong_count FROM wrong_question WHERE user_id = :userId', {
    userId });
  const trend = await query(
    `SELECT DATE(created_at) AS date, COUNT(*) AS count, ROUND(AVG(score),2) AS score
      FROM practice_record WHERE user_id = :userId GROUP BY DATE(created_at) ORDER BY date DESC LIMIT 14`,
    { userId }
  );
  const category = await query(
    `SELECT c.name_zh, c.name_en, c.name_vi, ROUND(AVG(r.score),2) AS score, COUNT(*) AS count
      FROM practice_record r JOIN term_category c ON r.category_id = c.id
      WHERE r.user_id = :userId AND r.category_id IS NOT NULL
      GROUP BY c.id ORDER BY score DESC`,
    { userId }
  );
  res.json({ summary: { ...summary[0], wrong_count: wrong[0].wrong_count }, trend: trend.reverse(),
    category });
  }));

analysisRouter.get('/admin', asyncHandler(async (req, res) => {
  const users = await query('SELECT COUNT(*) AS count FROM sys_user');
  const terms = await query('SELECT COUNT(*) AS count FROM railway_term');
  const practices = await query('SELECT COUNT(*) AS count, COALESCE(ROUND(AVG(score),2),0) AS avg_score
    FROM practice_record');
  const wrongTerms = await query(
    `SELECT t.term_zh, t.term_vi, SUM(w.wrong_count) AS wrong_count
      FROM wrong_question w JOIN railway_term t ON w.term_id = t.id
      GROUP BY t.id ORDER BY wrong_count DESC LIMIT 8`
  );
  const categories = await query(
    `SELECT c.name_zh, COUNT(t.id) AS term_count
      FROM term_category c LEFT JOIN railway_term t ON c.id = t.category_id
      GROUP BY c.id ORDER BY c.sort_order`
  );
  res.json({
    summary: {
      users: users[0].count,
      terms: terms[0].count,
      practices: practices[0].count,
      avg_score: practices[0].avg_score
    },
    wrongTerms,
    categories
  });
}));

```

3.22 server/src/routes/auth.js

文件路径：server/src/routes/auth.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { Router } from 'express';
import { env } from '../config/env.js';
import { query } from '../db/pool.js';
import { authRequired } from '../middleware/auth.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const authRouter = Router();

authRouter.post('/login', asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const ip = req.ip;
  const userAgent = req.headers['user-agent'] || '';
  const users = await query(

```

```

`SELECT u.*, r.code AS role_code, r.name_zh AS role_name_zh, r.name_en AS role_name_en, r.name_vi AS
role_name_vi
FROM sys_user u JOIN sys_role r ON u.role_id = r.id WHERE u.username = :username`,
{ username }
);
const user = users[0];
const ok = user && user.status === 'enabled' && await bcrypt.compare(password || '', user.password_hash);
await query(
  'INSERT INTO sys_login_log (username, ip, user_agent, success, message) VALUES (:username, :ip,
:userAgent, :success, :message)',
  { username: username || '', ip, userAgent, success: ok ? 1 : 0, message: ok ? '登录成功' : '用户名或密码错
误' }
);
if (!ok) return res.status(401).json({ message: '用户名或密码错误' });
const token = jwt.sign({ id: user.id, role: user.role_code }, env.jwtSecret, { expiresIn:
env.jwtExpiresIn });
res.json({
  token,
  user: {
    id: user.id,
    username: user.username,
    realName: user.real_name,
    email: user.email,
    roleCode: user.role_code,
    roleName: {
      zh: user.role_name_zh,
      en: user.role_name_en,
      vi: user.role_name_vi
    }
  }
});
});

authRouter.get('/profile', authRequired, asyncHandler(async (req, res) => {
  res.json(req.user);
}));

authRouter.put('/password', authRequired, asyncHandler(async (req, res) => {
  const { oldPassword, newPassword } = req.body;
  const rows = await query('SELECT password_hash FROM sys_user WHERE id = :id', { id: req.user.id });
  const ok = await bcrypt.compare(oldPassword || '', rows[0].password_hash);
  if (!ok) return res.status(400).json({ message: '原密码不正确' });
  const hash = await bcrypt.hash(newPassword, 10);
  await query('UPDATE sys_user SET password_hash = :hash WHERE id = :id', { hash, id: req.user.id });
  res.json({ message: 'ok' });
}));

authRouter.post('/logout', authRequired, asyncHandler(async (req, res) => {
  res.json({ message: 'ok' });
}));

```

3.23 server/src/routes/practice.js

文件路径：server/src/routes/practice.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```

import { Router } from 'express';
import { authRequired } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

export const practiceRouter = Router();
practiceRouter.use(authRequired);

function shuffle(items) {
  return [...items].sort(() => Math.random() - 0.5);
}

```

```

}

function buildQuestion(term, allTerms, type) {
  if (type === 'judge') {
    const shouldBeTrue = Math.random() > 0.35;
    const other = shuffle(allTerms.filter((item) => item.id !== term.id))[0] || term;
    return {
      termId: term.id,
      type,
      questionText: `${term.term_zh} = ${shouldBeTrue ? term.term_vi : other.term_vi}`,
      options: ['true', 'false'],
      correctAnswer: shouldBeTrue ? 'true' : 'false',
      explanation: term.definition_zh
    };
  }
  if (type === 'fill') {
    return {
      termId: term.id,
      type,
      questionText: `请填写中文术语“${term.term_zh}”对应的越南语翻译。`,
      options: [],
      correctAnswer: term.term_vi,
      explanation: term.definition_vi
    };
  }
  if (type === 'translate') {
    return {
      termId: term.id,
      type,
      questionText: `请翻译: ${term.term_vi}`,
      options: [],
      correctAnswer: term.term_zh,
      explanation: term.definition_zh
    };
  }
}

const options = shuffle([term, ...shuffle(allTerms.filter((item) => item.id !== term.id)).slice(0, 3)])
  .map((item) => item.term_vi);
return {
  termId: term.id,
  type: 'choice',
  questionText: `“${term.term_zh}” 的越南语翻译是?`,
  options,
  correctAnswer: term.term_vi,
  explanation: term.definition_zh
};
}

practiceRouter.post('/start', asyncHandler(async (req, res) => {
  const { categoryId = null, difficulty = null, count = 5, type = 'choice' } = req.body;
  const allTerms = await query(
    `SELECT * FROM railway_term
    WHERE status = 'enabled'
    AND (:categoryId IS NULL OR category_id = :categoryId)
    AND (:difficulty IS NULL OR difficulty = :difficulty)`,
    { categoryId, difficulty }
  );
  const selected = shuffle(allTerms).slice(0, Number(count));
  res.json({
    questions: selected.map((term) => buildQuestion(term, allTerms, type)),
    total: selected.length
  });
}));

practiceRouter.post('/submit', asyncHandler(async (req, res) => {
  const { type = 'choice', categoryId = null, difficulty = null, answers = [], durationSeconds = 0 } =
  req.body;
  let correct = 0;
  const normalized = answers.map((item) => {
    const isCorrect = String(item.userAnswer || '').trim().toLowerCase() === String(item.correctAnswer ||
    '').trim().toLowerCase();

```

```

    if (isCorrect) correct += 1;
    return { ...item, isCorrect };
  });
  const score = normalized.length ? Number(((correct / normalized.length) * 100).toFixed(2)) : 0;
  const result = await query(
    `INSERT INTO practice_record (user_id, practice_type, category_id, difficulty, total_count,
correct_count, score, duration_seconds)
    VALUES (:userId, :type, :categoryId, :difficulty, :total, :correct, :score, :durationSeconds)\`,
    { userId: req.user.id, type, categoryId, difficulty, total: normalized.length, correct, score,
durationSeconds }
  );
  for (const answer of normalized) {
    await query(
      `INSERT INTO practice_answer (record_id, term_id, question_type, question_text, user_answer,
correct_answer, is_correct)
      VALUES (:recordId, :termId, :questionType, :questionText, :userAnswer, :correctAnswer, :isCorrect)\`,
      {
        recordId: result.insertId,
        termId: answer.termId,
        questionType: answer.type,
        questionText: answer.questionText,
        userAnswer: answer.userAnswer,
        correctAnswer: answer.correctAnswer,
        isCorrect: answer.isCorrect ? 1 : 0
      }
    );
    if (!answer.isCorrect) {
      await query(
        `INSERT INTO wrong_question (user_id, term_id, wrong_count)
        VALUES (:userId, :termId, 1)
        ON DUPLICATE KEY UPDATE wrong_count = wrong_count + 1, last_wrong_at = CURRENT_TIMESTAMP\`,
        { userId: req.user.id, termId: answer.termId }
      );
    }
  }
  res.json({ recordId: result.insertId, total: normalized.length, correct, score, answers: normalized });
});

practiceRouter.get('/records', asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM practice_record WHERE user_id = :userId', {
userId: req.user.id });
  const rows = await query(`SELECT * FROM practice_record WHERE user_id = :userId ORDER BY created_at DESC
LIMIT ${limit} OFFSET ${offset}`, {
  userId: req.user.id
});
  res.json(paged(rows, total[0].count, page, pageSize));
}));

practiceRouter.get('/wrong-questions', asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM wrong_question WHERE user_id = :userId', {
userId: req.user.id });
  const rows = await query(
    `SELECT w.*, t.term_zh, t.term_en, t.term_vi, t.definition_zh, c.name_zh AS category_zh
    FROM wrong_question w
    JOIN railway_term t ON w.term_id = t.id
    JOIN term_category c ON t.category_id = c.id
    WHERE w.user_id = :userId
    ORDER BY w.last_wrong_at DESC
    LIMIT ${limit} OFFSET ${offset}`,
    { userId: req.user.id }
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));

```

3.24 server/src/routes/system.js

文件路径：server/src/routes/system.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
import bcrypt from 'bcryptjs';
import { Router } from 'express';
import { authRequired, requireRole } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

export const systemRouter = Router();
systemRouter.use(authRequired);

systemRouter.get('/roles', asyncHandler(async (req, res) => {
  if (!req.query.page && !req.query.pageSize) return res.json(await query('SELECT * FROM sys_role ORDER BY id'));
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_role');
  const rows = await query(`SELECT * FROM sys_role ORDER BY id LIMIT ${limit} OFFSET ${offset}`);
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.get('/users', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_user');
  const rows = await query(
    `SELECT u.id, u.username, u.real_name, u.email, u.status, u.created_at, r.code AS role_code, r.name_zh AS role_name
    FROM sys_user u JOIN sys_role r ON u.role_id = r.id ORDER BY u.id LIMIT ${limit} OFFSET ${offset}`
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.post('/users', requireRole('admin'), asyncHandler(async (req, res) => {
  const { username, password = '123456', real_name, email, role_id, status = 'enabled' } = req.body;
  const hash = await bcrypt.hash(password, 10);
  const result = await query(
    `INSERT INTO sys_user (username, password_hash, real_name, email, role_id, status) VALUES (:username, :hash, :real_name, :email, :role_id, :status)',
    { username, hash, real_name, email, role_id, status }
  );
  res.status(201).json({ id: result.insertId });
}));

systemRouter.put('/users/:id', requireRole('admin'), asyncHandler(async (req, res) => {
  await query(
    `UPDATE sys_user SET real_name=:real_name, email=:email, role_id=:role_id, status=:status WHERE id=:id`,
    { ...req.body, id: req.params.id }
  );
  res.json({ message: 'ok' });
}));

systemRouter.get('/logs/login', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_login_log');
  const rows = await query(`SELECT * FROM sys_login_log ORDER BY created_at DESC LIMIT ${limit} OFFSET ${offset}`);
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.get('/logs/operation', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_operation_log');
  const rows = await query(`SELECT * FROM sys_operation_log ORDER BY created_at DESC LIMIT ${limit} OFFSET ${offset}`);
  res.json(paged(rows, total[0].count, page, pageSize));
}));
```

文件路径：server/src/routes/terms.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
import { Router } from 'express';
import { authRequired, requireRole } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

export const termsRouter = Router();

termsRouter.use(authRequired);

termsRouter.get('/categories', asyncHandler(async (req, res) => {
  if (!req.query.page && !req.query.pageSize) {
    const rows = await query('SELECT * FROM term_category ORDER BY sort_order, id');
    return res.json(rows);
  }
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM term_category');
  const rows = await query(`SELECT * FROM term_category ORDER BY sort_order, id LIMIT ${limit} OFFSET ${offset}`);
  res.json(paged(rows, total[0].count, page, pageSize));
}));

termsRouter.post('/categories', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, name_vi, sort_order = 0, status = 'enabled' } = req.body;
  const result = await query(
    'INSERT INTO term_category (name_zh, name_en, name_vi, sort_order, status) VALUES (:name_zh, :name_en, :name_vi, :sort_order, :status)',
    { name_zh, name_en, name_vi, sort_order, status }
  );
  res.status(201).json({ id: result.insertId });
}));

termsRouter.put('/categories/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query(
    'UPDATE term_category SET name_zh=:name_zh, name_en=:name_en, name_vi=:name_vi, sort_order=:sort_order, status=:status WHERE id=:id',
    { ...req.body, id: req.params.id }
  );
  res.json({ message: 'ok' });
}));

termsRouter.delete('/categories/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query('DELETE FROM term_category WHERE id = :id', { id: req.params.id });
  res.json({ message: 'ok' });
}));

termsRouter.get('/', asyncHandler(async (req, res) => {
  const { keyword = '', categoryId, difficulty, status } = req.query;
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const params = {
    keyword: `%${keyword}%`,
    categoryId: categoryId || null,
    difficulty: difficulty || null,
    status: status || null,
    limit,
    offset
  };
  const total = await query(
    `SELECT COUNT(*) AS count
    FROM railway_term t JOIN term_category c ON t.category_id = c.id
    WHERE (:categoryId IS NULL OR t.category_id = :categoryId)
    AND (:difficulty IS NULL OR t.difficulty = :difficulty)
    AND (:status IS NULL OR t.status = :status)`
  );
  const rows = await query(
    `SELECT * FROM railway_term t JOIN term_category c ON t.category_id = c.id
    WHERE (:categoryId IS NULL OR t.category_id = :categoryId)
    AND (:difficulty IS NULL OR t.difficulty = :difficulty)
    AND (:status IS NULL OR t.status = :status)
    ORDER BY t.sort_order, t.id
    LIMIT :limit OFFSET :offset`
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));
```

```

        AND (t.term_zh LIKE :keyword OR t.term_en LIKE :keyword OR t.term_vi LIKE :keyword OR t.pinyin LIKE
:keyword)`,
        params
    );
    const rows = await query(
        `SELECT t.*, c.name_zh AS category_zh, c.name_en AS category_en, c.name_vi AS category_vi
        FROM railway_term t JOIN term_category c ON t.category_id = c.id
        WHERE (:categoryId IS NULL OR t.category_id = :categoryId)
            AND (:difficulty IS NULL OR t.difficulty = :difficulty)
            AND (:status IS NULL OR t.status = :status)
            AND (t.term_zh LIKE :keyword OR t.term_en LIKE :keyword OR t.term_vi LIKE :keyword OR t.pinyin LIKE
:keyword)
        ORDER BY t.updated_at DESC, t.id DESC
        LIMIT ${limit} OFFSET ${offset}`,
        params
    );
    res.json(paged(rows, total[0].count, page, pageSize));
});

termsRouter.post('/', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
    const data = { ...req.body, created_by: req.user.id };
    const result = await query(
        `INSERT INTO railway_term
        (category_id, term_zh, term_en, term_vi, pinyin, difficulty, definition_zh, definition_en,
definition_vi, example_zh, example_en, example_vi, status, created_by)
        VALUES (:category_id, :term_zh, :term_en, :term_vi, :pinyin, :difficulty, :definition_zh,
:definition_en, :definition_vi, :example_zh, :example_en, :example_vi, :status, :created_by)`,
        data
    );
    await query('INSERT INTO sys_operation_log (user_id, module, action, detail) VALUES (:userId, "terms",
"create", :detail)', {
        userId: req.user.id, detail: data.term_zh
    });
    res.status(201).json({ id: result.insertId });
});

termsRouter.put('/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
    await query(
        `UPDATE railway_term SET category_id=:category_id, term_zh=:term_zh, term_en=:term_en,
term_vi=:term_vi,
        pinyin=:pinyin, difficulty=:difficulty, definition_zh=:definition_zh, definition_en=:definition_en,
definition_vi=:definition_vi,
        example_zh=:example_zh, example_en=:example_en, example_vi=:example_vi, status=:status WHERE id=:id`,
        { ...req.body, id: req.params.id }
    );
    res.json({ message: 'ok' });
});

termsRouter.delete('/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
    await query('DELETE FROM railway_term WHERE id = :id', { id: req.params.id });
    res.json({ message: 'ok' });
});

termsRouter.post('/:id/favorite', asyncHandler(async (req, res) => {
    await query('INSERT IGNORE INTO favorite_term (user_id, term_id) VALUES (:userId, :termId)', { userId:
req.user.id, termId: req.params.id });
    res.json({ message: 'ok' });
});

```

3.26 server/src/utils/asyncHandler.js

文件路径：server/src/utils/asyncHandler.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

文件路径：server/src/utils/pagination.js

文件职责：该文件属于面向越南留学生的铁道专业汉越术语翻译学习软件V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
export function getPagination(query) {
  const page = Math.max(Number.parseInt(query.page || '1', 10), 1);
  const pageSize = Math.min(Math.max(Number.parseInt(query.pageSize || '10', 10), 1), 100);
  return {
    page,
    pageSize,
    limit: pageSize,
    offset: (page - 1) * pageSize
  };
}

export function paged(rows, total, page, pageSize) {
  return {
    rows,
    total: Number(total || 0),
    page,
    pageSize
  };
}
```

四.1 源码正文续页

README.md（续页 1）

文件路径：README.md

源码正文：

```
# chinese-vietnamese-railway-term-learning

面向越南留学生的铁道专业汉越术语翻译学习软件 V1.0

H5 前端 + Node.js 后台 + MySQL 数据库的铁道专业术语翻译学习平台。

## 功能范围

- 三语言界面：中文、English、Tiếng Việt
- 专业术语管理台：分类、三语术语、难度、释义、例句、启停用
- 练习平台：按分类/难度生成单选、判断、填空、翻译练习
- 数据分析：学习次数、正确率、错题、高频错误术语、分类统计
- 公共系统模块：登录、角色、用户、登录日志、操作日志

## 数据库配置

默认读取 `server/.env`：
```

```
DB_HOST=*** DB_PORT=3306 DB_USER=root DB_PASSWORD=*** DB_NAME=railway_term_learning JWT_SECRET=***
```

首次启动后端会自动创建数据表和演示数据。请先确保 MySQL 已启动，并且 `root` 账号允许从本机连接。

```
## 启动
```

npm install npm run dev

- 前端：http://***:5173
- 后端：http://***:3000

演示账号：

- 管理员：admin / ***
- 教师：teacher / ***

- 学生: student / *** 面向越南留学生的铁道专业汉越术语翻译学习软件V1.0源代码

开源协议

本项目使用 GNU General Public License v2.0 (GPL-2.0) 开源, 详见 `LICENSE`。

client/index.html (续页 1)

文件路径: client/index.html

源码正文:

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" href="/favicon.svg" />
    <title>汉越铁道术语学习软件</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

client/package.json (续页 1)

文件路径: client/package.json

源码正文:

```
{
  "name": "railway-term-learning-client",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build",
    "preview": "vite preview --host 0.0.0.0"
  },
  "dependencies": {
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.7",
    "vue": "^3.5.13"
  },
  "devDependencies": {}
}
```

client/src/App.vue (续页 1)

文件路径: client/src/App.vue

源码正文:

```
<script setup>
import { computed, onMounted, reactive, ref } from 'vue';
import { clearToken, request, setToken } from './api/http';
import { messages } from './locales/messages';

const lang = ref(localStorage.getItem('lang') || 'zh');
const page = ref('dashboard');
const loading = ref(false);
const error = ref('');
const loginForm = reactive({ username: 'admin', password: '***' });
const user = ref(JSON.parse(localStorage.getItem('user') || 'null'));
const terms = ref([]);
const categories = ref([]);
const users = ref([]);
const roles = ref([]);
const loginLogs = ref([]);
const adminAnalysis = ref(null);
```

```
const wrongQuestions = ref([]);
const filters = reactive({ keyword: '', categoryId: '', difficulty: '' });
const termPager = reactive({ page: 1, pageSize: 10, total: 0 });
const userPager = reactive({ page: 1, pageSize: 10, total: 0 });
const logPager = reactive({ page: 1, pageSize: 10, total: 0 });
const termForm = reactive(emptyTerm());
const editingId = ref(null);
const showTermForm = ref(false);
const practiceConfig = reactive({ type: 'choice', categoryId: '', difficulty: '', count: 5 });
const questions = ref([]);
const answers = reactive({});
const practiceResult = ref(null);

const t = computed(() => messages[lang.value]);
const isLoggedIn = computed(() => Boolean(user.value));
const canManage = computed(() => ['admin', 'teacher'].includes(user.value?.roleCode || user.value?.role_code));
const statsCards = computed(() => [
  { label: t.value.totalTerms, value: adminAnalysis.value?.summary?.terms ?? terms.value.length },
  { label: t.value.practiceCount, value: studentAnalysis.value?.summary?.practice_count ?? 0 },
  { label: t.value.avgScore, value: studentAnalysis.value?.summary?.avg_score ?? 0 },
  { label: t.value.correctRate, value: rateText.value }
]);
const rateText = computed(() => {
  const s = studentAnalysis.value?.summary;
  if (!s || Number(s.answer_count) === 0) return '0%';
  return `${Math.round((Number(s.correct_count) / Number(s.answer_count)) * 100)}%`;
});

function emptyTerm() {
  return {
    category_id: '',
    term_zh: '',
    term_en: '',
    term_vi: '',
    pinyin: '',
    difficulty: 'easy',
    definition_zh: '',
    definition_en: '',
    definition_vi: '',
    example_zh: '',
    example_en: '',
    example_vi: '',
    status: 'enabled'
  };
}

function setLanguage(value) {
  lang.value = value;
  localStorage.setItem('lang', value);
}

async function login() {
  error.value = '';
  loading.value = true;
  try {
    const data = await request('/auth/login', { method: 'POST', body: loginForm });
    setToken(data.token);
    user.value = data.user;
    localStorage.setItem('user', JSON.stringify(data.user));
    await bootstrap();
  } catch (err) {
    error.value = err.message;
  } finally {
    loading.value = false;
  }
}

function logout() {
```

```

clearToken();
user.value = null;
questions.value = [];
practiceResult.value = null;
}

async function bootstrap() {
  await Promise.all([loadCategories(), loadTerms(), loadAnalysis(), loadWrongQuestions()]);
  if (canManage.value) await Promise.all([loadUsers(), loadRoles(), loadLogs()]);
}

async function loadCategories() {
  categories.value = await request('/terms/categories');
}

async function loadTerms() {
  const params = new URLSearchParams();
  if (filters.keyword) params.set('keyword', filters.keyword);
  if (filters.categoryId) params.set('categoryId', filters.categoryId);
  if (filters.difficulty) params.set('difficulty', filters.difficulty);
  params.set('page', termPager.page);
  params.set('pageSize', termPager.pageSize);
  const data = await request(`/terms?${params.toString()}`);
  terms.value = data.rows || data;
  termPager.total = data.total ?? terms.value.length;
}

async function loadUsers() {
  const data = await request(`/system/users?page=${userPager.page}&pageSize=${userPager.pageSize}`);
  users.value = data.rows || data;
  userPager.total = data.total ?? users.value.length;
}

async function loadRoles() {
  roles.value = await request('/system/roles');
}

async function loadLogs() {
  const data = await request(`/system/logs/login?page=${logPager.page}&pageSize=${logPager.pageSize}`);
  loginLogs.value = data.rows || data;
  logPager.total = data.total ?? loginLogs.value.length;
}

async function loadAnalysis() {
  studentAnalysis.value = await request('/analysis/student');
  adminAnalysis.value = await request('/analysis/admin');
}

async function loadWrongQuestions() {
  const data = await request('/practice/wrong-questions?page=1&pageSize=20');
  wrongQuestions.value = data.rows || data;
}

function resetTermSearch() {

```

client/src/api/http.js (续页 1)

文件路径：client/src/api/http.js

源码正文：

```

const API_BASE = '/api';

export function getToken() {
  return localStorage.getItem('token');
}

export function setToken(token) {
  localStorage.setItem('token', token);
}

```

```

export function clearToken() {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
}

export async function request(path, options = {}) {
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
  const token = getToken();
  if (token) headers.Authorization = `Bearer ${token}`;
  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers,
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.message || 'Request failed');
  return data;
}

```

client/src/assets/styles.css (续页 1)

文件路径：client/src/assets/styles.css

源码正文：

```

:root {
  color: #172026;
  background: #f6f8fb;
  font-family: Inter, "Segoe UI", Arial, "Microsoft YaHei", sans-serif;
  font-size: 16px;
  letter-spacing: 0;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  border: 1px solid #cfd8e3;
  background: #ffffff;
  color: #172026;
  border-radius: 6px;
  padding: 9px 13px;
  cursor: pointer;
}

button.primary,
.primary {
  border-color: #0f766e;
  background: #0f766e;
  color: #ffffff;
}

button.active {
  border-color: #0f766e;
  background: #e7f6f3;
}

```

```
color: #0f766e;
}

input,
select,
textarea {
  width: 100%;
  border: 1px solid #cfd8e3;
  border-radius: 6px;
  background: #ffffff;
  padding: 10px 12px;
  color: #172026;
}

textarea {
  min-height: 74px;
  resize: vertical;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  padding: 13px 12px;
  border-bottom: 1px solid #e5eaf0;
  text-align: left;
  vertical-align: top;
}

th {
  color: #607080;
  font-size: 0.88rem;
  font-weight: 700;
  background: #f8fafc;
}

td small {
  display: block;
  color: #758292;
  margin-top: 4px;
}

.login-screen {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 24px;
  background: linear-gradient(145deg, #eef7f5 0%, #f7f9fb 52%, #fff4e8 100%);
}

.login-panel {
  width: min(460px, 100%);
  background: #ffffff;
  border: 1px solid #ele7ee;
  border-radius: 8px;
  padding: 34px;
  box-shadow: 0 18px 60px rgba(30, 44, 58, 0.12);
}

.login-panel h1 {
  margin: 6px 0 22px;
  font-size: 1.8rem;
  line-height: 1.22;
}

.eyebrow {
  margin: 0;
```

```
color: #0f766e;
font-weight: 700;
text-transform: uppercase;
font-size: 0.78rem;
}

.lang-row,
.toolbar,
.filters,
.practice-config,
.modal-actions {
  display: flex;
  gap: 10px;
  align-items: center;
  flex-wrap: wrap;
}

.lang-row {
  margin-bottom: 22px;
}

.form-stack {
  display: grid;
  gap: 14px;
}

label {
```

client/src/locales/messages.js (续页 1)

文件路径: client/src/locales/messages.js

源码正文:

```
export const messages = {
  zh: {
    app: '汉越铁道术语学习软件',
    login: '登录',
    logout: '退出',
    username: '用户名',
    password: '密码',
    dashboard: '学习看板',
    terms: '术语管理',
    practice: '练习平台',
    analysis: '数据分析',
    system: '系统管理',
    logs: '登录日志',
    categories: '分类',
    keyword: '关键词',
    addTerm: '新增术语',
    save: '保存',
    cancel: '取消',
    start: '开始练习',
    submit: '提交答案',
    wrongBook: '错题本',
    users: '用户管理',
    roles: '角色管理',
    totalTerms: '术语总数',
    avgScore: '平均得分',
    practiceCount: '练习次数',
    correctRate: '正确率'
  },
  en: {
    app: 'Chinese-Vietnamese Railway Term Learning',
    login: 'Login',
    logout: 'Logout',
    username: 'Username',
    password: 'Password',
    dashboard: 'Dashboard',
    terms: 'Terms',
    practice: 'Practice',
```

```
    analysis: 'Analytics',
    system: 'System',
    logs: 'Login Logs',
    categories: 'Categories',
    keyword: 'Keyword',
    addTerm: 'Add Term',
    save: 'Save',
    cancel: 'Cancel',
    start: 'Start Practice',
    submit: 'Submit',
    wrongBook: 'Wrong Book',
    users: 'Users',
    roles: 'Roles',
    totalTerms: 'Terms',
    avgScore: 'Average Score',
    practiceCount: 'Practice Count',
    correctRate: 'Accuracy'
  },
  vi: {
    app: 'Phần mềm học thuật ngữ đường sắt Trung-Việt',
    login: 'Đăng nhập',
    logout: 'Đăng xuất',
    username: 'Tên người dùng',
    password: 'Mật khẩu',
    dashboard: 'Bảng học tập',
    terms: 'Quản lý thuật ngữ',
    practice: 'Luyện tập',
    analysis: 'Phân tích dữ liệu',
    system: 'Quản trị hệ thống',
    logs: 'Nhật ký đăng nhập',
    categories: 'Danh mục',
    keyword: 'Từ khóa',
    addTerm: 'Thêm thuật ngữ',
    save: 'Lưu',
    cancel: 'Hủy',
    start: 'Bắt đầu luyện tập',
    submit: 'Nộp bài',
    wrongBook: 'Số câu sai',
    users: 'Quản lý người dùng',
    roles: 'Vai trò',
    totalTerms: 'Tổng thuật ngữ',
    avgScore: 'Điểm trung bình',
    practiceCount: 'Số lần luyện tập',
    correctRate: 'Tỷ lệ đúng'
  }
};
```

client/src/main.js (续页 1)

文件路径: client/src/main.js

源码正文:

```
import { createApp } from 'vue';
import App from './App.vue';
import './assets/styles.css';

createApp(App).mount('#app');
```

client/vite.config.js (续页 1)

文件路径: client/vite.config.js

源码正文:

```
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
```

```
    proxy: {
      '/api': 'http://***:3000'
    }
  }
});
```

package.json (续页 1)

文件路径: package.json

源码正文:

```
{
  "name": "chinese-vietnamese-railway-term-learning",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "面向越南留学生的铁道专业汉越术语翻译学习软件 V1.0",
  "scripts": {
    "dev": "concurrently \"npm run dev --workspace server\" \"npm run dev --workspace client\"",
    "build": "npm run build --workspace client",
    "start": "npm run start --workspace server",
    "install:all": "npm install --workspaces"
  },
  "workspaces": [
    "client",
    "server"
  ],
  "devDependencies": {
    "concurrently": "^9.1.2"
  }
}
```

server/.env.example (续页 1)

文件路径: server/.env.example

源码正文:

```
PORT=3000
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=railway_term_learning
JWT_SECRET=***
JWT_EXPIRES_IN=8h
```

server/package.json (续页 1)

文件路径: server/package.json

源码正文:

```
{
  "name": "railway-term-learning-server",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "nodemon src/index.js",
    "start": "node src/index.js",
    "seed:test-data": "node scripts/seed-test-data.js",
    "test:smoke": "node scripts/smoke-test.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "helmet": "^8.0.0",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.12.0",
  }
}
```

```

    "morgan": "^1.10.0",
    "nanoid": "^5.0.9",
    "zod": "^3.24.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}

```

server/scripts/seed-test-data.js (续页 1)

文件路径: server/scripts/seed-test-data.js

源码正文:

```

import bcrypt from 'bcryptjs';
import { initializeDatabase } from '../src/db/init.js';
import { getPool, query } from '../src/db/pool.js';

const MIN_ROWS = 100;

const zhTerms = ['铁路', '车站', '钢轨', '道岔', '机车', '客车', '信号机', '闭塞', '调度命令', '列车运行图'];
const viTerms = ['đường sắt', 'nhà ga', 'ray thép', 'ghí đường sắt', 'đầu máy', 'toa khách', 'tín hiệu', 'đóng đường', 'mệnh lệnh điều độ', 'biểu đồ chạy tàu'];
const enTerms = ['railway', 'station', 'rail', 'turnout', 'locomotive', 'coach', 'signal', 'block system', 'dispatching order', 'train diagram'];
const modules = ['auth', 'terms', 'practice', 'analysis', 'system'];
const actions = ['create', 'update', 'delete', 'query', 'export'];

function pad(num) {
  return String(num).padStart(3, '0');
}

async function count(table) {
  const rows = await query(`SELECT COUNT(*) AS count FROM ${table}`);
  return Number(rows[0].count);
}

async function ensureRoles() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT IGNORE INTO sys_role (code, name_zh, name_en, name_vi)
      VALUES (:code, :zh, :en, :vi)`,
      {
        code: `role_${pad(i)}`,
        zh: `测试角色${pad(i)}`,
        en: `Test Role ${pad(i)}`,
        vi: `Vai trò kiểm thử${pad(i)}`
      }
    );
  }
}

async function ensureUsers() {
  const hash = await bcrypt.hash('test123456', 10);
  const roles = await query('SELECT id FROM sys_role ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT IGNORE INTO sys_user (username, password_hash, real_name, email, role_id, status)
      VALUES (:username, :hash, :realName, :email, :roleId, 'enabled')`,
      {
        username: `test_user_${pad(i)}`,
        hash,
        realName: `测试用户${pad(i)}`,
        email: `test_user_${pad(i)}@example.com`,
        roleId: roles[(i - 1) % roles.length].id
      }
    );
  }
}

```

```
    for (let i = 1; i <= MIN_ROWS; i += 1) {
      await query(
        `INSERT INTO term_category (name_zh, name_en, name_vi, sort_order, status)
        SELECT :zh, :en, :vi, :sortOrder, 'enabled'
        WHERE NOT EXISTS (SELECT 1 FROM term_category WHERE name_zh = :zh)`,
        {
          zh: `测试分类${pad(i)}`,
          en: `Test Category ${pad(i)}`,
          vi: `Danh mục kiểm thử${pad(i)}`,
          sortOrder: 1000 + i
        }
      );
    }
  }
}

async function ensureTerms() {
  const categories = await query('SELECT id FROM term_category ORDER BY id LIMIT 100');
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    const idx = (i - 1) % zhTerms.length;
    await query(
      `INSERT INTO railway_term
      (category_id, term_zh, term_en, term_vi, pinyin, difficulty, definition_zh, definition_en,
definition_vi,
      example_zh, example_en, example_vi, status, created_by)
      SELECT :categoryId, :zh, :en, :vi, :pinyin, :difficulty, :defZh, :defEn, :defVi, :exZh, :exEn,
:exVi, 'enabled', 1
      WHERE NOT EXISTS (SELECT 1 FROM railway_term WHERE term_zh = :zh)`,
      {
        categoryId: categories[(i - 1) % categories.length].id,
        zh: `${zhTerms[idx]}测试${pad(i)}`,
        en: `${enTerms[idx]} test ${pad(i)}`,
        vi: `${viTerms[idx]} kiểm thử${pad(i)}`,
        pinyin: `pin yin ${pad(i)}`,
        difficulty: ['easy', 'medium', 'hard'][(i - 1) % 3],
        defZh: `测试术语${pad(i)}的中文释义。`,
        defEn: `English definition for test term ${pad(i)}.`,
        defVi: `Định nghĩa tiếng Việt cho thuật ngữ kiểm thử${pad(i)}.`,
        exZh: `这是测试术语${pad(i)}的示例句。`,
        exEn: `This is an example sentence for test term ${pad(i)}.`,
        exVi: `Đây là câu ví dụ cho thuật ngữ kiểm thử${pad(i)}.`
      }
    );
  }
}

async function ensureLoginLogs() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT INTO sys_login_log (username, ip, user_agent, success, message)
      SELECT :username, :ip, 'seed-test-agent', :success, :message
      WHERE (SELECT COUNT(*) FROM sys_login_log) < :minRows`,
      {
        username: `test_user_${pad(i)}`,
        ip: `*** % 200) + 1}`,
        success: i % 7 === 0 ? 0 : 1,
        message: i % 7 === 0 ? '测试登录失败' : '测试登录成功',
        minRows: MIN_ROWS
      }
    );
  }
}

async function ensureOperationLogs() {
  for (let i = 1; i <= MIN_ROWS; i += 1) {
    await query(
      `INSERT INTO sys_operation_log (user_id, module, action, detail)
      SELECT :userId, :module, :action, :detail
      WHERE (SELECT COUNT(*) FROM sys_operation_log) < :minRows`,
      {
```

userId: ((i - 1) % MIN_ROWS) + 1,

module: modules[(i - 1) % modules.length],

action: actions[(i - 1) % actions.length],

detail: `测试操作日志\${pad(i)}`,

minRows: MIN_ROWS

}

);

}

}

async function ensurePracticeRecords() {

const categories = await query('SELECT id FROM term_category ORDER BY id LIMIT 100');

for (let i = 1; i <= MIN_ROWS; i += 1) {

const total = 5 + (i % 6);

const correct = i % (total + 1);

await query(

`INSERT INTO practice_record (user_id, practice_type, category_id, difficulty, total_count, correct_count, score, duration_seconds)

SELECT :userId, :type, :categoryId, :difficulty, :total, :correct, :score, :duration

server/scripts/smoke-test.js (续页 1)

文件路径: server/scripts/smoke-test.js

源码正文:

const BASE_URL = process.env.BASE_URL || 'http://***:3000/api';

async function http(path, options = {}, token = '') {

const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };

if (token) headers.Authorization = `Bearer \${token}`;

const response = await fetch(`\${BASE\_URL}\${path}`, {

...options,

headers,

body: options.body ? JSON.stringify(options.body) : undefined

});

const text = await response.text();

const data = text ? JSON.parse(text) : {};

if (!response.ok) {

throw new Error(`\${options.method || 'GET'} \${path} failed: \${response.status} \${data.message || text}`);

}

return data;

}

function assert(condition, message) {

if (!condition) throw new Error(message);

}

function assertPage(data, page, pageSize, minTotal, name) {

assert(Array.isArray(data.rows), `\${name} rows missing`);

assert(data.page === page, `\${name} page mismatch`);

assert(data.pageSize === pageSize, `\${name} pageSize mismatch`);

assert(data.total >= minTotal, `\${name} total below \${minTotal}`);

assert(data.rows.length <= pageSize, `\${name} returned too many rows`);

}

async function main() {

const health = await http('/health');

assert(health.status === 'ok', 'health check failed');

const login = await http('/auth/login', {

method: 'POST',

body: { username: 'admin', password: '***' }

});

assert(login.token, 'login token missing');

const token = login.token;

const profile = await http('/auth/profile', {}, token);

assert(profile.username === 'admin', 'profile mismatch');

```

const categoryPage = await http('/terms/categories?page=2&pageSize=10', {}, token);
assertPage(categoryPage, 2, 10, 100, 'categories pagination');
const categories = await http('/terms/categories', {}, token);
assert(categories.length >= 100, 'category rows below 100');

const category = await http('/terms/categories', {
  method: 'POST',
  body: {
    name_zh: `接口测试分类${Date.now()}`,
    name_en: 'API Test Category',
    name_vi: 'Danh mục kiểm thử API',
    sort_order: 9999,
    status: 'enabled'
  }
}, token);
await http(`/terms/categories/${category.id}`, {
  method: 'PUT',
  body: {
    name_zh: `接口测试分类更新${Date.now()}`,
    name_en: 'API Test Category Updated',
    name_vi: 'Danh mục kiểm thử API cập nhật',
    sort_order: 9999,
    status: 'enabled'
  }
}, token);

const terms = await http('/terms?page=2&pageSize=10', {}, token);
assertPage(terms, 2, 10, 100, 'terms pagination');

const term = await http('/terms', {
  method: 'POST',
  body: {
    category_id: categories[0].id,
    term_zh: `接口测试术语${Date.now()}`,
    term_en: 'api test term',
    term_vi: 'thuật ngữ kiểm thử api',
    pinyin: 'jie kou ce shi shu yu',
    difficulty: 'easy',
    definition_zh: '接口测试中文释义',
    definition_en: 'API test English definition',
    definition_vi: 'Định nghĩa tiếng Việt kiểm thử API',
    example_zh: '接口测试示例句',
    example_en: 'API test example',
    example_vi: 'Ví dụ kiểm thử API',
    status: 'enabled'
  }
}, token);
await http(`/terms/${term.id}`, {
  method: 'PUT',
  body: {
    category_id: categories[0].id,
    term_zh: `接口测试术语更新${Date.now()}`,
    term_en: 'api test term updated',
    term_vi: 'thuật ngữ kiểm thử api cập nhật',
    pinyin: 'jie kou ce shi shu yu',
    difficulty: 'medium',
    definition_zh: '接口测试中文释义更新',
    definition_en: 'API test English definition updated',
    definition_vi: 'Định nghĩa tiếng Việt kiểm thử API cập nhật',
    example_zh: '接口测试示例句更新',
    example_en: 'API test example updated',
    example_vi: 'Ví dụ kiểm thử API cập nhật',
    status: 'enabled'
  }
}, token);
await http(`/terms/${term.id}/favorite`, { method: 'POST' }, token);
await http(`/terms/${term.id}`, { method: 'DELETE' }, token);
await http(`/terms/categories/${category.id}`, { method: 'DELETE' }, token);

const practice = await http('/practice/start', {

```

```

    method: 'POST',
    body: { type: 'choice', count: 5, categoryId: null, difficulty: null }
  }, token);
assert(practice.questions.length > 0, 'practice question generation failed');
const result = await http('/practice/submit', {
  method: 'POST',
  body: {
    type: 'choice',
    categoryId: null,
    difficulty: null,
    answers: practice.questions.map((question) => ({
      ...question,
      userAnswer: question.correctAnswer
    })))
  }
}, token);
assert(Number(result.score) === 100, 'practice scoring failed');

const records = await http('/practice/records?page=1&pageSize=10', {}, token);
assertPage(records, 1, 10, 1, 'practice records pagination');
const wrongPage = await http('/practice/wrong-questions?page=1&pageSize=10', {}, token);
assertPage(wrongPage, 1, 10, 1, 'wrong questions pagination');

const studentAnalysis = await http('/analysis/student', {}, token);
assert(studentAnalysis.summary, 'student analysis missing');
const adminAnalysis = await http('/analysis/admin', {}, token);

```

server/src/config/env.js (续页 1)

文件路径：server/src/config/env.js

源码正文：

```

import dotenv from 'dotenv';

dotenv.config();

export const env = {
  port: Number(process.env.PORT || 3000),
  db: {
    host: process.env.DB_HOST || '***',
    port: Number(process.env.DB_PORT || 3306),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '***',
    database: process.env.DB_NAME || 'railway_term_learning'
  },
  jwtSecret: process.env.JWT_SECRET || '***',
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '8h'
};

```

server/src/db/init.js (续页 1)

文件路径：server/src/db/init.js

源码正文：

```

import bcrypt from 'bcryptjs';
import { ensureDatabase, getPool, query } from './pool.js';

const schema = [
  `CREATE TABLE IF NOT EXISTS sys_role (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(40) NOT NULL UNIQUE,
    name_zh VARCHAR(80) NOT NULL,
    name_en VARCHAR(80) NOT NULL,
    name_vi VARCHAR(80) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
  `CREATE TABLE IF NOT EXISTS sys_user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(120) NOT NULL,

```

```

email VARCHAR(120),
role_id INT NOT NULL,
status ENUM('enabled','disabled') DEFAULT 'enabled',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (role_id) REFERENCES sys_role(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS sys_login_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  ip VARCHAR(80),
  user_agent VARCHAR(255),
  success TINYINT(1) NOT NULL,
  message VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS sys_operation_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  module VARCHAR(60) NOT NULL,
  action VARCHAR(60) NOT NULL,
  detail VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS term_category (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name_zh VARCHAR(100) NOT NULL,
  name_en VARCHAR(100) NOT NULL,
  name_vi VARCHAR(100) NOT NULL,
  sort_order INT DEFAULT 0,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS railway_term (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  term_zh VARCHAR(160) NOT NULL,
  term_en VARCHAR(160) NOT NULL,
  term_vi VARCHAR(160) NOT NULL,
  pinyin VARCHAR(200),
  difficulty ENUM('easy','medium','hard') DEFAULT 'easy',
  definition_zh TEXT,
  definition_en TEXT,
  definition_vi TEXT,
  example_zh TEXT,
  example_en TEXT,
  example_vi TEXT,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_by INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES term_category(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS practice_record (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  practice_type VARCHAR(40) NOT NULL,
  category_id INT,
  difficulty VARCHAR(20),
  total_count INT NOT NULL,
  correct_count INT NOT NULL,
  score DECIMAL(5,2) NOT NULL,
  duration_seconds INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS practice_answer (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  record_id BIGINT NOT NULL,
  term_id INT NOT NULL,

```

```

question_type VARCHAR(40) NOT NULL,
question_text VARCHAR(255) NOT NULL,
user_answer VARCHAR(255),
correct_answer VARCHAR(255) NOT NULL,
is_correct TINYINT(1) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (record_id) REFERENCES practice_record(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS wrong_question (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  wrong_count INT DEFAULT 1,
  last_wrong_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_user_term (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`,
`CREATE TABLE IF NOT EXISTS favorite_term (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_fav (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`
];

```

```

const categories = [
  ['铁路基础设施', 'Railway Infrastructure', 'Cơ sở hạ tầng đường sắt', 1],
  ['轨道工程', 'Track Engineering', 'Kỹ thuật đường ray', 2],
  ['机车车辆', 'Rolling Stock', 'Đầu máy toa xe', 3],
  ['信号与通信', 'Signaling and Communication', 'Tín hiệu và thông tin', 4],
  ['调度指挥', 'Dispatching Command', 'Điều độ chỉ huy', 5]
];

```

```

const terms = [
  [1, '铁路', 'railway', 'đường sắt', 'tie lu', 'easy', '供列车运行的交通系统。', 'A transport system for train operation.', 'Hệ thống giao thông dành cho tàu chạy.', '铁路连接城市与港口。', 'The railway connects cities and ports.', 'Đường sắt kết nối thành phố và cảng.'],
  [1, '车站', 'station', 'nhà ga', 'che zhan', 'easy', '办理旅客乘降或货物作业的地点。', 'A place for passenger boarding or freight operations.', 'Nơi phục vụ hành khách lên xuống hoặc xếp dỡ hàng hóa.', '列车将在下一车站停车。', 'The train will stop at the next station.', 'Tàu sẽ dừng ở nhà ga tiếp theo.'],
  [2, '钢轨', 'rail', 'ray thép', 'gang gui', 'medium', '承受车轮荷载并引导车辆运行的轨道部件。', 'A track component that bears wheel loads and guides vehicles.', 'Bộ phận đường ray chịu tải bánh xe và dẫn hướng phương tiện.', '钢轨需要定期检查。', 'Rails require regular inspection.', 'Ray thép cần được kiểm tra định kỳ.'],
  [2, '道岔', 'turnout', 'ghi đường sắt', 'dao cha', 'hard', '使列车从一股轨道转入另一股轨道的设备。', 'Equipment that allows trains to move from one track to another.', 'Thiết bị cho phép tàu chuyển từ đường ray này sang đường ray khác.', '道岔状态影响行车安全。', 'Turnout status affects operation safety.', 'Trạng thái ghi ảnh hưởng đến an toàn chạy tàu.'],
  [3, '机车', 'locomotive', 'đầu máy', 'ji che', 'medium', '提供牵引动力的铁路车辆。', 'A railway vehicle that provides traction power.', 'Phương tiện đường sắt cung cấp lực kéo.', '电力机车适合高速牵引。', 'Electric locomotives suit high-speed traction.', 'Đầu máy điện phù hợp với kéo tàu tốc độ cao.'],
  [3, '客车', 'passenger coach', 'toa khách', 'ke che', 'easy', '用于运送旅客的铁路车辆。', 'A railway vehicle used to carry passengers.', 'Toa xe đường sắt dùng để chở hành khách.', '客车内部配有座椅。', 'Passenger coaches are equipped with seats.', 'Toa khách được trang bị ghế ngồi.'],
  [4, '信号机', 'signal', 'tín hiệu đường sắt', 'xin hao ji', 'medium', '向司机显示行车条件的信号设备。', 'A device that displays movement authority to drivers.', 'Thiết bị hiển thị điều kiện chạy tàu cho lái tàu.', '司机必须确认信号机显示。', 'Drivers must confirm the signal aspect.', 'Lái tàu phải xác nhận hiển thị tín hiệu.'],
  [4, '闭塞', 'block system', 'hệ thống đóng đường', 'bi se', 'hard', '保证区间内列车安全间隔的行车制度。', 'An operating system that ensures safe train separation.', 'Chế độ vận hành bảo đảm khoảng cách an toàn giữa các tàu.', '自动闭塞提高线路通过能力。', 'Automatic block systems improve line capacity.', 'Đóng đường tự động nâng cao năng lực thông qua tuyến.'],
  [5, '调度命令', 'dispatching order', 'mệnh lệnh điều độ', 'diao du ming ling', 'hard', '调度员发布的行车组织指令。', 'An operation instruction issued by a dispatcher.', 'Chỉ lệnh tổ chức chạy tàu do điều độ viên ban hành.', '调度命令必须准确传达。', 'Dispatching orders must be communicated accurately.', 'Mệnh lệnh điều độ phải được truyền đạt chính xác.'],
  [5, '列车运行图', 'train diagram', 'biểu đồ chạy tàu', 'lie che yun xing tu', 'hard', '规定列车运行时间和顺序的技术文件。', 'A technical document defining train times and sequence.', 'Tài liệu kỹ thuật quy định thời gian và thứ tự chạy tàu.', '运行图是调度工作的基础。', 'The train diagram is the basis of dispatching.', 'Biểu đồ chạy tàu là cơ sở của công tác điều độ.'],

```

```
];
```

```
export async function initializeDatabase() {
  await ensureDatabase();
  const pool = getPool();
  for (const sql of schema) await pool.query(sql);

  const roleCount = await query('SELECT COUNT(*) AS count FROM sys_role');
  if (roleCount[0].count === 0) {
    await query(`INSERT INTO sys_role (code, name_zh, name_en, name_vi) VALUES`
```

server/src/db/pool.js (续页 1)

文件路径：server/src/db/pool.js

源码正文：

```
import mysql from 'mysql2/promise';
import { env } from '../config/env.js';

let pool;

export async function ensureDatabase() {
  const server = await mysql.createConnection({
    host: env.db.host,
    port: env.db.port,
    user: env.db.user,
    password: env.db.password,
    multipleStatements: true
  });
  await server.query(`CREATE DATABASE IF NOT EXISTS \`${env.db.database}\` DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci`);
  await server.end();
}

export function getPool() {
  if (!pool) {
    pool = mysql.createPool({
      ...env.db,
      waitForConnections: true,
      connectionLimit: 10,
      namedPlaceholders: true,
      timezone: '+08:00'
    });
  }
  return pool;
}

export async function query(sql, params = {}) {
  const [rows] = await getPool().execute(sql, params);
  return rows;
}
```

server/src/db/schema.sql (续页 1)

文件路径：server/src/db/schema.sql

源码正文：

```
CREATE TABLE IF NOT EXISTS sys_role (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(40) NOT NULL UNIQUE,
  name_zh VARCHAR(80) NOT NULL,
  name_en VARCHAR(80) NOT NULL,
  name_vi VARCHAR(80) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS sys_user (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(120) NOT NULL,
```

```
real_name VARCHAR(80) NOT NULL,
email VARCHAR(120),
role_id INT NOT NULL,
status ENUM('enabled','disabled') DEFAULT 'enabled',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (role_id) REFERENCES sys_role(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS sys_login_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  ip VARCHAR(80),
  user_agent VARCHAR(255),
  success TINYINT(1) NOT NULL,
  message VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS sys_operation_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  module VARCHAR(60) NOT NULL,
  action VARCHAR(60) NOT NULL,
  detail VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS term_category (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name_zh VARCHAR(100) NOT NULL,
  name_en VARCHAR(100) NOT NULL,
  name_vi VARCHAR(100) NOT NULL,
  sort_order INT DEFAULT 0,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS railway_term (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  term_zh VARCHAR(160) NOT NULL,
  term_en VARCHAR(160) NOT NULL,
  term_vi VARCHAR(160) NOT NULL,
  pinyin VARCHAR(200),
  difficulty ENUM('easy','medium','hard') DEFAULT 'easy',
  definition_zh TEXT,
  definition_en TEXT,
  definition_vi TEXT,
  example_zh TEXT,
  example_en TEXT,
  example_vi TEXT,
  status ENUM('enabled','disabled') DEFAULT 'enabled',
  created_by INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES term_category(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS practice_record (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  practice_type VARCHAR(40) NOT NULL,
  category_id INT,
  difficulty VARCHAR(20),
  total_count INT NOT NULL,
  correct_count INT NOT NULL,
  score DECIMAL(5,2) NOT NULL,
  duration_seconds INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS practice_answer (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  record_id BIGINT NOT NULL,
  term_id INT NOT NULL,
  question_type VARCHAR(40) NOT NULL,
  question_text VARCHAR(255) NOT NULL,
  user_answer VARCHAR(255),
  correct_answer VARCHAR(255) NOT NULL,
  is_correct TINYINT(1) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (record_id) REFERENCES practice_record(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS wrong_question (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  wrong_count INT DEFAULT 1,
  last_wrong_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_user_term (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS favorite_term (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  term_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_fav (user_id, term_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

server/src/index.js (续页 1)

文件路径：server/src/index.js

源码正文：

```

import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import { env } from './config/env.js';
import { initializeDatabase } from './db/init.js';
import { authRouter } from './routes/auth.js';
import { termsRouter } from './routes/terms.js';
import { practiceRouter } from './routes/practice.js';
import { analysisRouter } from './routes/analysis.js';
import { systemRouter } from './routes/system.js';

const app = express();

app.use(helmet());
app.use(cors());
app.use(express.json({ limit: '2mb' }));
app.use(morgan('dev'));

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', name: 'railway-term-learning-server' });
});
app.use('/api/auth', authRouter);
app.use('/api/terms', termsRouter);
app.use('/api/practice', practiceRouter);
app.use('/api/analysis', analysisRouter);
app.use('/api/system', systemRouter);

app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ message: err.message || 'Internal Server Error' });
});

```

```

initializeDatabase()
  .then(() => {
    app.listen(env.port, '0.0.0.0', () => {
      console.log(`Server listening on http://**:${env.port}`);
    });
  })
  .catch((error) => {
    console.error('Database initialization failed:', error.message);
    process.exit(1);
  });

```

server/src/middleware/auth.js (续页 1)

文件路径: server/src/middleware/auth.js

源码正文:

```

import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { query } from '../db/pool.js';

export async function authRequired(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    if (!token) return res.status(401).json({ message: 'Unauthorized' });
    const payload = jwt.verify(token, env.jwtSecret);
    const users = await query(
      `SELECT u.id, u.username, u.real_name, u.email, u.status, r.code AS role_code, r.name_zh AS role_name
      FROM sys_user u JOIN sys_role r ON u.role_id = r.id WHERE u.id = :id`,
      { id: payload.id }
    );
    if (!users.length || users[0].status !== 'enabled') return res.status(401).json({ message: 'Unauthorized' });
    req.user = users[0];
    next();
  } catch (error) {
    res.status(401).json({ message: 'Unauthorized' });
  }
}

export function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role_code)) return res.status(403).json({ message: 'Forbidden' });
    next();
  };
}

```

server/src/routes/analysis.js (续页 1)

文件路径: server/src/routes/analysis.js

源码正文:

```

import { Router } from 'express';
import { authRequired } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const analysisRouter = Router();
analysisRouter.use(authRequired);

analysisRouter.get('/student', asyncHandler(async (req, res) => {
  const userId = req.query.userId || req.user.id;
  const summary = await query(
    `SELECT COUNT(*) AS practice_count,
      COALESCE(SUM(total_count),0) AS answer_count,
      COALESCE(SUM(correct_count),0) AS correct_count,
      COALESCE(ROUND(AVG(score),2),0) AS avg_score
      FROM practice_record WHERE user_id = :userId`,
    { userId }
  );
});

```

```

const wrong = await query('SELECT COUNT(*) AS wrong_count FROM wrong_question WHERE user_id = :userId', {
  userId });
const trend = await query(
  `SELECT DATE(created_at) AS date, COUNT(*) AS count, ROUND(AVG(score),2) AS score
    FROM practice_record WHERE user_id = :userId GROUP BY DATE(created_at) ORDER BY date DESC LIMIT 14`,
  { userId }
);
const category = await query(
  `SELECT c.name_zh, c.name_en, c.name_vi, ROUND(AVG(r.score),2) AS score, COUNT(*) AS count
    FROM practice_record r JOIN term_category c ON r.category_id = c.id
    WHERE r.user_id = :userId AND r.category_id IS NOT NULL
    GROUP BY c.id ORDER BY score DESC`,
  { userId }
);
res.json({ summary: { ...summary[0], wrong_count: wrong[0].wrong_count }, trend: trend.reverse(),
category });
});

analysisRouter.get('/admin', asyncHandler(async (req, res) => {
  const users = await query('SELECT COUNT(*) AS count FROM sys_user');
  const terms = await query('SELECT COUNT(*) AS count FROM railway_term');
  const practices = await query('SELECT COUNT(*) AS count, COALESCE(ROUND(AVG(score),2),0) AS avg_score
FROM practice_record');
  const wrongTerms = await query(
    `SELECT t.term_zh, t.term_vi, SUM(w.wrong_count) AS wrong_count
      FROM wrong_question w JOIN railway_term t ON w.term_id = t.id
      GROUP BY t.id ORDER BY wrong_count DESC LIMIT 8`
  );
  const categories = await query(
    `SELECT c.name_zh, COUNT(t.id) AS term_count
      FROM term_category c LEFT JOIN railway_term t ON c.id = t.category_id
      GROUP BY c.id ORDER BY c.sort_order`
  );
  res.json({
    summary: {
      users: users[0].count,
      terms: terms[0].count,
      practices: practices[0].count,
      avg_score: practices[0].avg_score
    },
    wrongTerms,
    categories
  });
}));

```

server/src/routes/auth.js (续页 1)

文件路径：server/src/routes/auth.js

源码正文：

```

import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { Router } from 'express';
import { env } from '../config/env.js';
import { query } from '../db/pool.js';
import { authRequired } from '../middleware/auth.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const authRouter = Router();

authRouter.post('/login', asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const ip = req.ip;
  const userAgent = req.headers['user-agent'] || '';
  const users = await query(
    `SELECT u.*, r.code AS role_code, r.name_zh AS role_name_zh, r.name_en AS role_name_en, r.name_vi AS
role_name_vi
    FROM sys_user u JOIN sys_role r ON u.role_id = r.id WHERE u.username = :username`,
    { username }
  );
  );

```

```

const user = users[0];
const ok = user && user.status === 'enabled' && await bcrypt.compare(password || '', user.password_hash);
await query(
  'INSERT INTO sys_login_log (username, ip, user_agent, success, message) VALUES (:username, :ip,
:userAgent, :success, :message)',
  { username: username || '', ip, userAgent, success: ok ? 1 : 0, message: ok ? '登录成功' : '用户名或密码错误' }
);
if (!ok) return res.status(401).json({ message: '用户名或密码错误' });
const token = jwt.sign({ id: user.id, role: user.role_code }, env.jwtSecret, { expiresIn:
env.jwtExpiresIn });
res.json({
  token,
  user: {
    id: user.id,
    username: user.username,
    realName: user.real_name,
    email: user.email,
    roleCode: user.role_code,
    roleName: {
      zh: user.role_name_zh,
      en: user.role_name_en,
      vi: user.role_name_vi
    }
  }
});
});

authRouter.get('/profile', authRequired, asyncHandler(async (req, res) => {
  res.json(req.user);
}));

authRouter.put('/password', authRequired, asyncHandler(async (req, res) => {
  const { oldPassword, newPassword } = req.body;
  const rows = await query('SELECT password_hash FROM sys_user WHERE id = :id', { id: req.user.id });
  const ok = await bcrypt.compare(oldPassword || '', rows[0].password_hash);
  if (!ok) return res.status(400).json({ message: '原密码不正确' });
  const hash = await bcrypt.hash(newPassword, 10);
  await query('UPDATE sys_user SET password_hash = :hash WHERE id = :id', { hash, id: req.user.id });
  res.json({ message: 'ok' });
}));

authRouter.post('/logout', authRequired, asyncHandler(async (req, res) => {
  res.json({ message: 'ok' });
}));

```

server/src/routes/practice.js (续页 1)

文件路径：server/src/routes/practice.js

源码正文：

```

import { Router } from 'express';
import { authRequired } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

export const practiceRouter = Router();
practiceRouter.use(authRequired);

function shuffle(items) {
  return [...items].sort(() => Math.random() - 0.5);
}

function buildQuestion(term, allTerms, type) {
  if (type === 'judge') {
    const shouldBeTrue = Math.random() > 0.35;
    const other = shuffle(allTerms.filter((item) => item.id !== term.id))[0] || term;
    return {
      termId: term.id,

```

```

    type,
    questionText: `${term.term_zh} = ${shouldBeTrue ? term.term_vi : other.term_vi}`,
    options: ['true', 'false'],
    correctAnswer: shouldBeTrue ? 'true' : 'false',
    explanation: term.definition_zh
  };
}
if (type === 'fill') {
  return {
    termId: term.id,
    type,
    questionText: `请填写中文术语“${term.term_zh}”对应的越南语翻译。`,
    options: [],
    correctAnswer: term.term_vi,
    explanation: term.definition_vi
  };
}
if (type === 'translate') {
  return {
    termId: term.id,
    type,
    questionText: `请翻译: ${term.term_vi}`,
    options: [],
    correctAnswer: term.term_zh,
    explanation: term.definition_zh
  };
}
const options = shuffle([term, ...shuffle(allTerms.filter((item) => item.id !== term.id)).slice(0, 3)])
  .map((item) => item.term_vi);
return {
  termId: term.id,
  type: 'choice',
  questionText: `“${term.term_zh}” 的越南语翻译是?`,
  options,
  correctAnswer: term.term_vi,
  explanation: term.definition_zh
};
}

practiceRouter.post('/start', asyncHandler(async (req, res) => {
  const { categoryId = null, difficulty = null, count = 5, type = 'choice' } = req.body;
  const allTerms = await query(
    `SELECT * FROM railway_term
    WHERE status = 'enabled'
    AND (:categoryId IS NULL OR category_id = :categoryId)
    AND (:difficulty IS NULL OR difficulty = :difficulty)`,
    { categoryId, difficulty }
  );
  const selected = shuffle(allTerms).slice(0, Number(count));
  res.json({
    questions: selected.map((term) => buildQuestion(term, allTerms, type)),
    total: selected.length
  });
}));

practiceRouter.post('/submit', asyncHandler(async (req, res) => {
  const { type = 'choice', categoryId = null, difficulty = null, answers = [], durationSeconds = 0 } =
  req.body;
  let correct = 0;
  const normalized = answers.map((item) => {
    const isCorrect = String(item.userAnswer || '').trim().toLowerCase() === String(item.correctAnswer ||
    '').trim().toLowerCase();
    if (isCorrect) correct += 1;
    return { ...item, isCorrect };
  });
  const score = normalized.length ? Number(((correct / normalized.length) * 100).toFixed(2)) : 0;
  const result = await query(
    `INSERT INTO practice_record (user_id, practice_type, category_id, difficulty, total_count,
    correct_count, score, duration_seconds)
    VALUES (:userId, :type, :categoryId, :difficulty, :total, :correct, :score, :durationSeconds)`,

```

```

    { userId: req.user.id, type, categoryId, difficulty, total: normalized.length, correct, score,
      durationSeconds }
    );
    for (const answer of normalized) {
      await query(
        `INSERT INTO practice_answer (record_id, term_id, question_type, question_text, user_answer,
correct_answer, is_correct)
        VALUES (:recordId, :termId, :questionType, :questionText, :userAnswer, :correctAnswer, :isCorrect)`,
        {
          recordId: result.insertId,
          termId: answer.termId,
          questionType: answer.type,
          questionText: answer.questionText,
          userAnswer: answer.userAnswer,
          correctAnswer: answer.correctAnswer,
          isCorrect: answer.isCorrect ? 1 : 0
        }
      );
      if (!answer.isCorrect) {
        await query(
          `INSERT INTO wrong_question (user_id, term_id, wrong_count)
          VALUES (:userId, :termId, 1)
          ON DUPLICATE KEY UPDATE wrong_count = wrong_count + 1, last_wrong_at = CURRENT_TIMESTAMP`,
          { userId: req.user.id, termId: answer.termId }
        );
      }
    }
    res.json({ recordId: result.insertId, total: normalized.length, correct, score, answers: normalized });
  });

practiceRouter.get('/records', asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM practice_record WHERE user_id = :userId', {
    userId: req.user.id });
  const rows = await query(`SELECT * FROM practice_record WHERE user_id = :userId ORDER BY created_at DESC
LIMIT ${limit} OFFSET ${offset}`, {
    userId: req.user.id
  });
  res.json(paged(rows, total[0].count, page, pageSize));
}));

practiceRouter.get('/wrong-questions', asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM wrong_question WHERE user_id = :userId', {
    userId: req.user.id });
  const rows = await query(
    `SELECT w.*, t.term_zh, t.term_en, t.term_vi, t.definition_zh, c.name_zh AS category_zh
    FROM wrong_question w
    JOIN railway_term t ON w.term_id = t.id
    JOIN term_category c ON t.category_id = c.id
    WHERE w.user_id = :userId
    ORDER BY w.last_wrong_at DESC
    LIMIT ${limit} OFFSET ${offset}`,
    { userId: req.user.id }
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));

```

server/src/routes/system.js (续页 1)

文件路径: server/src/routes/system.js

源码正文:

```

import bcrypt from 'bcryptjs';
import { Router } from 'express';
import { authRequired, requireRole } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

```

```

export const systemRouter = Router();
systemRouter.use(authRequired);

systemRouter.get('/roles', asyncHandler(async (req, res) => {
  if (!req.query.page && !req.query.pageSize) return res.json(await query('SELECT * FROM sys_role ORDER BY id'));
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_role');
  const rows = await query('SELECT * FROM sys_role ORDER BY id LIMIT ${limit} OFFSET ${offset}');
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.get('/users', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_user');
  const rows = await query(
    `SELECT u.id, u.username, u.real_name, u.email, u.status, u.created_at, r.code AS role_code, r.name_zh AS role_name
    FROM sys_user u JOIN sys_role r ON u.role_id = r.id ORDER BY u.id LIMIT ${limit} OFFSET ${offset}`
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.post('/users', requireRole('admin'), asyncHandler(async (req, res) => {
  const { username, password = '123456', real_name, email, role_id, status = 'enabled' } = req.body;
  const hash = await bcrypt.hash(password, 10);
  const result = await query(
    `INSERT INTO sys_user (username, password_hash, real_name, email, role_id, status) VALUES (:username, :hash, :real_name, :email, :role_id, :status)',
    { username, hash, real_name, email, role_id, status }
  );
  res.status(201).json({ id: result.insertId });
}));

systemRouter.put('/users/:id', requireRole('admin'), asyncHandler(async (req, res) => {
  await query(
    `UPDATE sys_user SET real_name=:real_name, email=:email, role_id=:role_id, status=:status WHERE id=:id`,
    { ...req.body, id: req.params.id }
  );
  res.json({ message: 'ok' });
}));

systemRouter.get('/logs/login', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_login_log');
  const rows = await query('SELECT * FROM sys_login_log ORDER BY created_at DESC LIMIT ${limit} OFFSET ${offset}');
  res.json(paged(rows, total[0].count, page, pageSize));
}));

systemRouter.get('/logs/operation', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM sys_operation_log');
  const rows = await query('SELECT * FROM sys_operation_log ORDER BY created_at DESC LIMIT ${limit} OFFSET ${offset}');
  res.json(paged(rows, total[0].count, page, pageSize));
}));

```

server/src/routes/terms.js (续页 1)

文件路径: server/src/routes/terms.js

源码正文:

```

import { Router } from 'express';
import { authRequired, requireRole } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { asyncHandler } from '../utils/asyncHandler.js';
import { getPagination, paged } from '../utils/pagination.js';

```

```
termsRouter.use(authRequired);
```

```
termsRouter.get('/categories', asyncHandler(async (req, res) => {
  if (!req.query.page && !req.query.pageSize) {
    const rows = await query('SELECT * FROM term_category ORDER BY sort_order, id');
    return res.json(rows);
  }
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const total = await query('SELECT COUNT(*) AS count FROM term_category');
  const rows = await query(`SELECT * FROM term_category ORDER BY sort_order, id LIMIT ${limit} OFFSET ${offset}`);
  res.json(paged(rows, total[0].count, page, pageSize));
}));
```

```
termsRouter.post('/categories', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, name_vi, sort_order = 0, status = 'enabled' } = req.body;
  const result = await query(
    'INSERT INTO term_category (name_zh, name_en, name_vi, sort_order, status) VALUES (:name_zh, :name_en, :name_vi, :sort_order, :status)',
    { name_zh, name_en, name_vi, sort_order, status }
  );
  res.status(201).json({ id: result.insertId });
}));
```

```
termsRouter.put('/categories/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query(
    'UPDATE term_category SET name_zh=:name_zh, name_en=:name_en, name_vi=:name_vi, sort_order=:sort_order, status=:status WHERE id=:id',
    { ...req.body, id: req.params.id }
  );
  res.json({ message: 'ok' });
}));
```

```
termsRouter.delete('/categories/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query('DELETE FROM term_category WHERE id = :id', { id: req.params.id });
  res.json({ message: 'ok' });
}));
```

```
termsRouter.get('/', asyncHandler(async (req, res) => {
  const { keyword = '', categoryId, difficulty, status } = req.query;
  const { page, pageSize, limit, offset } = getPagination(req.query);
  const params = {
    keyword: `%${keyword}%`,
    categoryId: categoryId || null,
    difficulty: difficulty || null,
    status: status || null,
    limit,
    offset
  };
  const total = await query(
    `SELECT COUNT(*) AS count
    FROM railway_term t JOIN term_category c ON t.category_id = c.id
    WHERE (:categoryId IS NULL OR t.category_id = :categoryId)
    AND (:difficulty IS NULL OR t.difficulty = :difficulty)
    AND (:status IS NULL OR t.status = :status)
    AND (t.term_zh LIKE :keyword OR t.term_en LIKE :keyword OR t.term_vi LIKE :keyword OR t.pinyin LIKE :keyword)`,
    params
  );
  const rows = await query(
    `SELECT t.*, c.name_zh AS category_zh, c.name_en AS category_en, c.name_vi AS category_vi
    FROM railway_term t JOIN term_category c ON t.category_id = c.id
    WHERE (:categoryId IS NULL OR t.category_id = :categoryId)
    AND (:difficulty IS NULL OR t.difficulty = :difficulty)
    AND (:status IS NULL OR t.status = :status)
    AND (t.term_zh LIKE :keyword OR t.term_en LIKE :keyword OR t.term_vi LIKE :keyword OR t.pinyin LIKE :keyword)
    ORDER BY t.updated_at DESC, t.id DESC`
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));
```

```

    params
  );
  res.json(paged(rows, total[0].count, page, pageSize));
}));

termsRouter.post('/', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  const data = { ...req.body, created_by: req.user.id };
  const result = await query(
    `INSERT INTO railway_term
      (category_id, term_zh, term_en, term_vi, pinyin, difficulty, definition_zh, definition_en,
      definition_vi, example_zh, example_en, example_vi, status, created_by)
      VALUES (:category_id, :term_zh, :term_en, :term_vi, :pinyin, :difficulty, :definition_zh,
      :definition_en, :definition_vi, :example_zh, :example_en, :example_vi, :status, :created_by)`,
    data
  );
  await query('INSERT INTO sys_operation_log (user_id, module, action, detail) VALUES (:userId, "terms",
"create", :detail)', {
    userId: req.user.id, detail: data.term_zh
  });
  res.status(201).json({ id: result.insertId });
}));

termsRouter.put('/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query(
    `UPDATE railway_term SET category_id=:category_id, term_zh=:term_zh, term_en=:term_en,
term_vi=:term_vi,
    pinyin=:pinyin, difficulty=:difficulty, definition_zh=:definition_zh, definition_en=:definition_en,
definition_vi=:definition_vi,
    example_zh=:example_zh, example_en=:example_en, example_vi=:example_vi, status=:status WHERE id=:id`,
    { ...req.body, id: req.params.id }
  );
  res.json({ message: 'ok' });
}));

termsRouter.delete('/:id', requireRole('admin', 'teacher'), asyncHandler(async (req, res) => {
  await query('DELETE FROM railway_term WHERE id = :id', { id: req.params.id });
  res.json({ message: 'ok' });
}));

termsRouter.post('/:id/favorite', asyncHandler(async (req, res) => {
  await query('INSERT IGNORE INTO favorite_term (user_id, term_id) VALUES (:userId, :termId)', { userId:
req.user.id, termId: req.params.id });
  res.json({ message: 'ok' });
}));

```

server/src/utils/asyncHandler.js (续页 1)

文件路径: server/src/utils/asyncHandler.js

源码正文:

```
export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

server/src/utils/pagination.js (续页 1)

文件路径: server/src/utils/pagination.js

源码正文:

```

export function getPagination(query) {
  const page = Math.max(Number.parseInt(query.page || '1', 10), 1);
  const pageSize = Math.min(Math.max(Number.parseInt(query.pageSize || '10', 10), 1), 100);
  return {
    page,
    pageSize,
    limit: pageSize,
    offset: (page - 1) * pageSize
  };
}

export function paged(rows, total, page, pageSize) {

```

```
    return {
      rows,
      total: Number(total || 0),
      page,
      pageSize
    };
  }
}
```